

INSTRUCTOR GUIDE — CONFIDENTIAL

PostgreSQL Database Fundamentals

with Performance & Tuning

Organised by A UK Trade and Infrastructure Enterprise Limited (AUKTIE)
Coventry Conferences, The Techno Centre, Coventry University Technology Park
Puma Way, Coventry CV1 2TT, United Kingdom | 22nd June – 3rd July 2026

Lead Instructor & Author

Dr Tertsegha Joseph Anande

PhD, MSc, MBCS, FHEA

Lecturer in Computing & Deputy Programme Leader, QAHE Limited / Ulster University —
Birmingham, UK

Specialist in Machine Learning, AutoML, Ensemble Learning, Database Systems, and Cyber-Physical Security. Dr Anande brings both academic research rigour and industry-facing teaching practice to this programme, combining PostgreSQL database engineering with the data governance and digital transformation context required for national-scale social protection systems.

Profile & publications: drtertseghaanande.com

LinkedIn: linkedin.com/in/tertsegha-anande-phd-msc-fhea-mbcs-71939520a

Important

CONFIDENTIAL — Lead Instructor Use Only

PROGRAMME SCHEDULE UPDATE: Monday 22 June has been REMOVED from the programme.
The programme now opens on TUESDAY 23 JUNE.

Participants settle in from 09:30; training begins at 10:00.

Daily timetable: 10:00 start; coffee 11:30–11:45; lunch 13:00–14:00; coffee 15:30–15:45; close 17:00.

Monday 29 June: Mid-Training Assessment Day (proctor 10:00–13:00; Lead Instructor joins 14:00).

Innovation Hub, Birmingham study visit: Friday 26 June.

Monday 29 June is now a Mid-Training Assessment Day (practical tasks only, proctor-supervised in the morning; Lead Instructor joins at 14:00) rather than a teaching day.

The Innovation Hub, Birmingham study visit has moved from Thursday 25 June to Friday 26 June.

This guide covers all of the above plus the five remaining technical teaching days.

Contains full session plans, demonstration scripts, model answers, and facilitation notes.

Do not distribute to participants — except the Mid-Training Assessment proctor instructions, which should be shared with the proctor in advance.

Programme theme

Strengthening Enterprise Data Management, Digital Government Systems and
Social Protection Delivery through PostgreSQL and International Best Practice

Day	Date	Theme / activity	Technical focus
W1 Mon	22 Jun	REMOVED — Programme now begins on Tuesday 23	Monday 22 June has been removed from the programme schedule

		June	
W1 Tue	23 Jun	Foundations + Database Design & Architecture (merged)	DPI, PG ecosystem, process/memory/storage, WAL, MVCC, normalisation, schema, environment setup
W1 Wed	24 Jun	SQL Fundamentals & Advanced Querying	DML, DDL, JOINS, subqueries, window functions, reporting
W1 Thu	25 Jun	PostgreSQL Administration, Security & Backup	RBAC, RLS, security best practices, backup, disaster recovery
W1 Fri	26 Jun	Study Visit — Innovation Hub, Birmingham	Local government digital transformation & citizen services
W2 Mon	29 Jun	Mid-Training Assessment Day	Practical tasks only; proctor AM, Lead Instructor from 14:00
W2 Tue	30 Jun	Performance Tuning & Database Optimisation	EXPLAIN ANALYZE, indexing, partitioning, monitoring, capacity planning
W2 Wed	1 Jul	ETL/ELT & Data Governance (merged)	Migration, validation, dedup, NDP Act 2023, lineage, MDM, interoperability, NSR workshop
W2 Fri	3 Jul	Programme Review & Closing Ceremony (afternoon)	Group blueprint presentations, study visit reflections, certificates, closing remarks

Intermediate learners: Focus on core syntax, concepts and guided exercises.

Advanced learners: Add challenge tasks, tuning rationale, and architectural design decisions.

How to use this guide

Each day opens with an Instructor Brief (preparation, timing, tone guidance).

Sessions are divided into: Conceptual Content, Demonstrations (run live on projector), and Lab Exercises.

Bridge phrases connect every technical concept to Digital Public Infrastructure and NSR context.

Model answers are provided for all exercises.

Challenge tasks are marked clearly — assign to advanced learners without slowing the rest of the group.

Learning outcomes at the end of each day match the official AUKTIE programme objectives.

APPENDIX — PRE-SESSION SETUP GUIDE (PARTICIPANT-FACING)

Distribute to all participants before Monday 22 / Tuesday 23 June

Instructor note

The following pages are the exact setup guide sent to participants ahead of the programme. Monday 22 June (optional) and the Tuesday 23 June setup-verification slot are both opportunities to use it. Use it during registration to assist anyone whose setup did not complete.

All exercises across the programme assume: database 'lab_nsr', superuser 'postgres', password 'postgres'.

If a participant arrives with a different configuration, the fastest fix is usually:

```
ALTER USER postgres PASSWORD 'postgres'; then CREATE DATABASE lab_nsr;
```

PRE-SESSION SETUP GUIDE

Software installation through your first psql connection — complete before Day 1 (Tue 23 June)

Note

Complete this guide on your own laptop BEFORE the programme begins on Tuesday 23 June.

Participants settle in from 09:30; training starts at 10:00.

It takes approximately 30-45 minutes depending on your internet connection.

If you get stuck, arrive by 09:30 for registration (settle-in from 09:30, training starts 10:00) for assisted setup.

No prior database experience is required.

Which PostgreSQL version should I install?

This programme is written against PostgreSQL 18 — the current stable release — and all exercises are tested against it. If you can install PostgreSQL 18, do so.

If you already have PostgreSQL 15, 16, or 17 installed on your laptop (for example, from previous work or another course), you do NOT need to uninstall it.

Every exercise in this programme runs without modification on versions 15-18.

Wherever a step is version-specific, this guide shows the PG18 command first, followed by an 'If you are on PG15-17' note where anything differs.

The only features that are PG18-only are explicitly marked 'PG18+' in the instructor guide — these are optional 'what's new' demonstrations, not required exercises.

What you will install

Software	What it is	Why you need it
PostgreSQL 18 (or 15-17)	The database server itself	The actual database engine used throughout the programme
psql	Command-line client (installed with PostgreSQL)	The primary tool used in every session
pgAdmin 4	Graphical interface (bundled on Windows)	Optional visual tool for browsing tables and data

Overview of the process

1. Install PostgreSQL 18 (includes psql and pgAdmin 4) — or use an existing 15-17 install
2. Verify the installation
3. Connect to PostgreSQL using psql for the first time
4. Create the training database and run your first command

Jump to the section for your operating system: Windows, macOS, or Ubuntu/Linux. All platforms then converge on the shared 'First connection' section.

WINDOWS 10 / 11 — INSTALLATION

1 Download the PostgreSQL 18 installer

5. Open your web browser and go to:

<https://www.postgresql.org/download/windows/>

6. Click 'Download the installer' — this takes you to the EDB download page

7. Find PostgreSQL 18.x (the latest 18.x version) for Windows x86-64 and download it (~300 MB)

Tip

If your internet connection is slow, ask the registration desk for a USB drive with an offline copy of the installer.

Already have PostgreSQL 15, 16, or 17 installed? You can skip this download entirely — your existing installation works for every exercise in this programme. Go straight to Step 5 to confirm your superuser password, then continue to 'First connection'.

2 Run the installer

8. Double-click the downloaded .exe file. If Windows shows a security warning, click Yes

9. Click Next on the welcome screen

10. Installation Directory: leave as default and click Next

3 Select components

Ensure ALL FOUR boxes are checked: PostgreSQL Server, pgAdmin 4, Stack Builder (can uncheck), Command Line Tools. Click Next.

4 Choose the data directory

11. Leave the default data directory and click Next

5 Set the superuser password — IMPORTANT

Important

For this training programme, set the password to: `postgres`

Use exactly this password (all lowercase). Every exercise assumes this password.
WRITE THIS DOWN — you will need it every time you connect.
(In production you would choose a strong unique password — 'postgres' is used here only to keep the training consistent for everyone.)

If you already have PostgreSQL 15-17 installed with a DIFFERENT password, either remember your existing password for use throughout the programme, or run: `ALTER USER postgres PASSWORD 'postgres';` from psql to standardise it.

12. Type 'postgres' in the Password field, then again in Retype Password, then click Next

6 Set the port number

13. Leave the port as the default: 5432, click Next

Tip

If you already have an older PostgreSQL version installed and running on port 5432, the installer will suggest a different port (e.g. 5433) for the new instance. Either is fine — note which port you used, as you will need it when connecting:
`psql -U postgres -p 5433`

7 Choose the locale

14. Leave the locale as default (or 'English, United Kingdom'), click Next

8 Complete the installation

15. Review the summary and click Next. Installation takes 2-5 minutes

Important

UNCHECK 'Launch Stack Builder at exit' before clicking Finish.
Stack Builder is not needed for this programme.

16. Click Finish

Checkpoint — confirm before continuing

PostgreSQL 18 (or your existing 15-17 installation) is ready.
pgAdmin 4 appears in your Start Menu.
Continue to 'First connection' below.

macOS — INSTALLATION

Works for both Intel and Apple Silicon (M1/M2/M3) Macs

Method A (Postgres.app) is simpler and recommended. Method B (Homebrew) is for existing Homebrew users.

Tip

Already have PostgreSQL 15, 16, or 17 installed and working? You do not need to upgrade — every exercise in this programme works on versions 15-18. Skip to Step 4 (Postgres.app) or the last command (Homebrew) below to confirm your superuser password, then go to 'First connection'. Just substitute your existing version number where '18' appears below.

Method A — Postgres.app (recommended)

1 Download and install

17. Go to <https://postgresapp.com> and click Download (~200 MB .dmg)
18. Double-click the .dmg, drag Postgres.app into Applications, then open it
19. If macOS blocks it: System Settings -> Privacy & Security -> 'Open Anyway'

2 Initialise the server

20. Click 'Initialize' or '+' to create a new server
21. This creates a PostgreSQL 18 server on port 5432 — the elephant icon appears in the menu bar

Tip

Postgres.app lets you run multiple versions side by side. If you already have a 15-17 server initialised, you can keep using that one — just note its port number (shown in the Postgres.app window) for the connection step.

3 Add psql to your PATH

22. Open Terminal (Cmd+Space, type 'Terminal') and run:

```
echo 'export PATH="/Applications/Postgres.app/Contents/Versions/18/bin:$PATH"' >>
 ~/.zshrc
```

Important

If you are using an existing PostgreSQL 15, 16, or 17 server instead, replace '18' in the path above with your version number, e.g. .../Versions/16/bin

23. Close and reopen Terminal, then verify:

```
psql --version
```


Tip

Expected: psql (PostgreSQL) 18.x (or 15.x/16.x/17.x if using an existing install) —
if 'command not found', restart your computer and retry.

4**Create the postgres superuser**

```
psql -U $(whoami) -d postgres
CREATE ROLE postgres WITH SUPERUSER LOGIN PASSWORD 'postgres';
\q
```

Important

Use the training password: `postgres` — every exercise assumes this.

If this role already exists from a previous install, run instead:

```
ALTER ROLE postgres WITH PASSWORD 'postgres';
```

Method B — Homebrew (for existing Homebrew users)

```
brew install postgresql@18
brew services start postgresql@18
echo 'export PATH="/opt/homebrew/opt/postgresql@18/bin:$PATH"' >> ~/.zshrc
# restart Terminal, then:
psql --version
psql -d postgres -c "CREATE ROLE postgres WITH SUPERUSER LOGIN PASSWORD 'postgres';"
```

Tip

Already have postgresql@15, @16, or @17 via Homebrew? Just run:

```
brew services start postgresql@16
```

(substitute your version)

then the same psql --version and CREATE ROLE commands above.

Checkpoint — confirm before continuing

PostgreSQL is installed and running (version 18, or your existing 15-17).

Continue to 'First connection' below.

UBUNTU / DEBIAN LINUX — INSTALLATION

Tip

Already have postgresql-15, -16, or -17 installed and running? Skip to Step 4 to confirm the superuser password, then go to 'First connection'. Every exercise in this programme works on versions 15-18.

1 Add the PostgreSQL apt repository

```
sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg
main" \
> /etc/apt/sources.list.d/pgdg.list'
wget --quiet -O - https://www.postgresql.org/media/keys/ACCC4CF8.asc | \
sudo gpg --dearmor -o /usr/share/keyrings/postgresql.gpg
sudo apt update
```

2 Install PostgreSQL 18 and pgAdmin

```
sudo apt install -y postgresql-18 postgresql-client-18
sudo apt install -y pgadmin4-desktop # optional
```

Tip

To install an older supported version instead (e.g. if your distro's repo lags behind, or you prefer to match an existing production system), replace '18' with 15, 16, or 17 in the command above — e.g. postgresql-16 postgresql-client-16

3 Start the service

```
sudo systemctl start postgresql
sudo systemctl enable postgresql
sudo systemctl status postgresql # look for 'active (running)', press q to exit
```

4 Set the postgres user password

```
sudo -u postgres psql -c "ALTER USER postgres PASSWORD 'postgres';"
```

Important

Use the training password: **postgres** — every exercise assumes this.

Checkpoint — confirm before continuing

PostgreSQL (18, or your existing 15-17) is installed and running.
Continue to 'First connection' below.

FIRST CONNECTION — ALL OPERATING SYSTEMS

Verifying installation and connecting to psql for the first time

Operating system	How to open a terminal
Windows	Press the Windows key, type 'Command Prompt'. Or search for 'SQL Shell (psql)' for a pre-configured shortcut.
macOS	Press Cmd+Space, type 'Terminal', press Enter
Linux	Press Ctrl+Alt+T, or open Terminal from your applications menu

Step 1 — Connect to PostgreSQL

```
psql -U postgres
```

- If prompted 'Password for user postgres:', type postgres (characters won't be visible) and press Enter

Tip

Using 'SQL Shell (psql)' on Windows: press Enter to accept the four default prompts (Server/Database/Port/Username), then enter the password.

Expected result

```
postgres=#
```

Checkpoint — confirm before continuing

Seeing 'postgres=#' confirms you are connected — PostgreSQL is waiting for SQL commands.

Troubleshooting

Error message	Likely cause	Fix
'psql' is not recognized / command not found	psql not in PATH	Windows: use 'SQL Shell (psql)'. macOS: re-check PATH step. Restart terminal after PATH changes.
could not connect to server: Connection refused	Service not running	Windows: Services app -> start 'postgresql-x64-18' (or your version). macOS: open Postgres.app. Linux: sudo systemctl start postgresql
password authentication failed	Password not set to 'postgres'	Re-run the password-setting step for your OS
role 'postgres' does not exist	macOS: role not created	psql -U \$(whoami) -d postgres, then CREATE ROLE postgres

		WITH SUPERUSER LOGIN PASSWORD 'postgres';
--	--	--

Step 2 — Run your first commands

Every SQL command ends with a semicolon (;). Commands starting with backslash (\) are psql meta-commands and do not need one.

2.1 — Check the PostgreSQL version

```
SELECT version();
```

Tip

Expected: PostgreSQL 18.x on x86_64-... — confirms PostgreSQL 18 is running.

If you are using an existing 15, 16, or 17 install, you will see that version number instead — for example 'PostgreSQL 16.4 on x86_64-...'. This is completely fine. Every exercise in this programme runs correctly on PostgreSQL 15 through 18. Make a note of YOUR version number — your instructor may ask for it during Day 1 so the room knows which version is in use at each station.

2.2 — List existing databases

```
\l
```

2.3 — Check your connection identity

```
SELECT current_user, current_database(), now();
```

Step 3 — Create the training database

```
CREATE DATABASE lab_nsr;
```

Tip

Expected: CREATE DATABASE

3.1 — Connect to it

```
\c lab_nsr
```

Tip

Your prompt changes from postgres=# to lab_nsr=#

3.2 — Install extensions used during the programme

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS pgstattuple;
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

Important

If pgaudit or pg_stat_statements error mentioning 'shared_preload_libraries', don't worry — this needs a config change + restart, demonstrated in the Week 1 Friday session. Skip for now. pgcrypto and pgstattuple should install cleanly on all platforms.

3.3 — Verify

```
SELECT extname, extversion FROM pg_extension ORDER BY extname;
```

Step 4 — Exit psql

```
\q
```

Final checklist — confirm before Tuesday

#	Item	Done?
1	PostgreSQL is installed (version 18 recommended; 15-17 also fully supported)	
2	I can run 'psql -U postgres' and connect with password 'postgres'	
3	SELECT version(); returns PostgreSQL 15.x, 16.x, 17.x, or 18.x — I have noted which	
4	The lab_nsr database has been created	
5	I can connect to it with \c lab_nsr	
6	pgcrypto and pgstattuple extensions are installed	
7	I know how to exit psql using \q	

Ready for Tuesday

If all seven items are checked, your environment is ready for the programme.

Bring your laptop charger — sessions run 10:00–17:00 with breaks.

If anything failed, arrive by 09:30 Tuesday for assisted setup.

Keep this guide accessible during Day 1. Training starts at 10:00; arrive by 09:30 to settle in.

WEEK 1
MON 22
JUN

Optional Day — Registration, Environment Setup & Orientation

Status: REMOVED — Monday 22 June is no longer part of the programme schedule

Format: Light-touch only. No content here is a prerequisite for Tuesday — everything is briefly recapped.

Important

SCHEDULE CHANGE — read before preparing this session

Most participants arrive in the UK on the evening of Monday 22 June and will not be present.

This day exists **ONLY** for participants who happen to be available earlier. Attendance may be partial, uncertain, or zero — do not plan around a fixed headcount.

Nothing taught today creates a dependency for Tuesday. Tuesday 23 June opens with its own brief recap and a mandatory setup-verification slot, so latecomers are not disadvantaged.

Keep energy light and welcoming — this is a soft start, not a compressed teaching day.

If literally no one is available, skip this day entirely; nothing is lost.

Suggested structure (flex freely — there is no fixed timetable)

Time	Session
Whenever ready	Informal registration as participants arrive — no fixed start time
~30-45 mins	Welcome and introductions (keep this relaxed — repeat names won't hurt)
~45-60 mins	Environment setup: install PostgreSQL, connect, create lab_nsr (see Setup Guide)
~30-45 mins	Light orientation: programme objectives, theme, global digital government context
Remainder	Open Q&A, informal conversation, early finish if the group is small

Registration and welcome

Treat this as a soft opening. Some participants may trickle in across the day depending on flight connections. Greet each arrival individually rather than waiting for a fixed start time.

Environment setup (the most valuable use of this day)

Instructor note

If participants are available Monday, this is the best possible use of the time:

getting PostgreSQL installed and the lab_nsr database created BEFORE the pressure of Tuesday's dense merged day. Anyone who completes this Monday can skip Tuesday's setup-verification slot entirely and move straight into content.

Use the Pre-Session Setup Guide (included in this pack) exactly as written.

Confirm each participant reaches: `SELECT version();` returning PostgreSQL 15-18, and `CREATE DATABASE lab_nsr;` succeeding.

Light orientation content (optional, time-permitting)

Opening framing — say this or adapt it

- Nigeria is at a pivotal moment in its digital government journey. The systems you build will serve millions of citizens. PostgreSQL is not a technical choice — it is a strategic choice for scale, openness, and sovereignty over your own data infrastructure.
- Walk through the AUKTIE programme theme: 'Strengthening Enterprise Data Management, Digital Government Systems and Social Protection Delivery.'
- Walk through the 12 official programme objectives at a high level — detail comes later in the week.

Global trends in digital government — light touch

- Digital Public Infrastructure (DPI): the three foundational layers — digital identity (e.g. NIN/Aadhaar), payment rails (e.g. CBN NIBSS), and data exchange platforms (e.g. NSR)
- Why PostgreSQL for government: open-source (no licence cost), used by India Aadhaar (1.4B records), UK DWP Universal Credit (22M claimants), Kenya Huduma Namba
- Ask informally: 'What is the single biggest data management problem in your organisation today?' Note answers — these feed into Tuesday's opening discussion too, so don't worry if Monday's group is small or this gets repeated.

Tip

Whatever is covered today, plan to give it a genuine but brief (5 minute) recap on Tuesday rather than assuming it 'sticks' — some Monday attendees will be jet-lagged, and Tuesday-only arrivals need the same framing fresh. Repetition here costs little and protects against an uneven start to the technical week.

Learning outcome

For those present: a head start on environment setup and an early, relaxed introduction to the programme. No assessment, no required content, no dependency for Tuesday.

WEEK 1
TUE 23
JUN

Enterprise Data Transformation, PostgreSQL Foundations & Architecture

Official theme: Merged day — Enterprise Data Transformation & PostgreSQL Foundations + Database Design & PostgreSQL Architecture

Programme objectives addressed: #1 Database Administration | #3 Enterprise Database Architecture | #9 Social Protection Systems | #11 NSR Modernisation | #12 Public Sector Digital Transformation

Important

SCHEDULE CHANGE — read before preparing this session

Most participants arrive in the UK on the evening of Monday 22 June and travel directly to Coventry, so THIS is the first day most of the group attends. Monday 22 June ran as an optional light day (registration/setup/orientation) for anyone who happened to be available — treat that as a bonus, not a baseline. Assume a MIXED ROOM: some participants did setup yesterday, some are arriving completely fresh today.

This single day MERGES the original Monday (Foundations) and Tuesday (Architecture) content. It is necessarily a long, dense day — pace deliberately and protect the environment-setup time at the end; nothing later in the week works without it.

If the room is visibly fatigued by mid-afternoon, the Schema Development practical (Session 4) can be shortened to one group exercise instead of three — do not cut the WAL/MVCC demonstrations or the environment setup.

Instructor note

Arrive 45 minutes early. Verify all participant laptops. Participant settling-in (09:30–10:00) is Use 09:30–10:00 to assist with any remaining setup issues.

The opening question sets the tone for the whole programme — start with a question, not slides.

Identify the room's technical range early and calibrate the schema workshop depth accordingly.

Write participant answers on the flipchart and keep it visible all week.

Session 1 — Registration, setup check, brief recap & architecture 09:00–10:30

Registration and introductions 10:00–10:15

Circulate during registration. Learn names. Note who attended Monday's optional session (and therefore already has PostgreSQL installed) versus who is arriving completely fresh — this determines how you run the setup-verification slot below.

Mandatory setup-verification slot 10:15–10:35

Important

Do not skip this even if most of the room attended Monday. Some participants are guaranteed to be arriving fresh today with no PostgreSQL installation at all. Run this as a fast, parallel-track exercise, not a lecture.

24. Ask: 'Hands up if you already have PostgreSQL installed and can run SELECT version();' — this splits the room
25. Those WITH a working setup: ask them to pull up their lab_nsr database and wait — or pair them with someone who needs help
26. Those WITHOUT: walk them through the Pre-Session Setup Guide quick-path (Windows/macOS/Linux install, password 'postgres', CREATE DATABASE lab_nsr) while the rest wait or help
27. Target: every single participant has run SELECT version(); and CREATE DATABASE lab_nsr; successfully by 10:35. This is non-negotiable — nothing later in the day works without it

Brief recap for latecomers 10:35–10:50

Instructor note

This is a 15-minute RECAP, not a re-teach. If most of the room attended Monday, move through this briskly and let Monday attendees confirm/nod along rather than re-explaining everything from scratch. If most of the room is fresh, give it the full 15. Either way, keep it to the headlines below — depth comes through the rest of the day.

- One-line programme framing: 'Strengthening Enterprise Data Management, Digital Government Systems and Social Protection Delivery through PostgreSQL.'
- Digital Public Infrastructure (DPI) in one breath: digital identity (NIN) + payment rails (CBN NIBSS) + data exchange (NSR)
- Why PostgreSQL for government: open-source, SQL standard, used by India Aadhaar (1.4B records), UK DWP Universal Credit (22M claimants)
- Ask (or re-confirm from Monday): 'What is the single biggest data management problem in your organisation today?' Write answers on the flipchart — keep this visible all week

Session 2 — Introduction to PostgreSQL & ecosystem overview 09:50–10:30

Conceptual content (20 mins)

- PostgreSQL 18 (the current stable release): async I/O improvements, OAuth 2.0 authentication support, continued query parallelism and security enhancements. PostgreSQL 15-17 remain fully supported and in widespread production use — every concept in this programme applies unchanged across versions 15-18
- Client tools: psql (command line), pgAdmin 4 (GUI), DBeaver (alternative GUI)
- Key components: server process (postmaster), backend processes (one per connection), background workers
- Extensions ecosystem: pgAudit (compliance logging), pgcrypto (encryption), pgstattuple (storage analysis), PostGIS (spatial)

DEMO 1.1 — First PostgreSQL connection (20 mins)

```
-- Connect to the server (everyone should already be connected from the setup slot)
psql -U postgres

-- Check version
```

```

SELECT version();

-- List databases
\l

-- Confirm lab_nsr exists and connect
\c lab_nsr

-- Who are we connected as?
SELECT current_user, current_database(), now();

-- What extensions are available?
SELECT name, default_version
FROM   pg_available_extensions
WHERE  name IN ('pgaudit', 'pgcrypto', 'pgstattuple', 'pg_stat_statements')
ORDER BY name;

```

Session 3 — PostgreSQL architecture: process, memory, storage, WAL, MVCC 11:45–13:00

Process architecture (20 mins)

THEORY — The client-server process model and why PostgreSQL chose it

PostgreSQL uses a PROCESS-PER-CONNECTION architecture, sometimes called the 'multi-process model'. This is a deliberate, decades-old design decision, not an accident.

When the server starts, one process — the POSTMASTER — is created. It does three things: it listens on a TCP/IP port (default 5432) and a Unix socket; it manages shared memory (shared_buffers, WAL buffers, lock tables) that all other processes will use; and it forks a new BACKEND PROCESS every time a client connects.

Forking a full OS process (rather than spawning a lightweight thread, as MySQL and SQL Server do) means each connection gets a fully isolated memory space. If one backend crashes due to a bug or corrupted data, it cannot corrupt another backend's memory — the operating system's process isolation protects every other connection. The postmaster detects the crash, and as a precaution restarts ALL backends to guarantee shared memory is not silently corrupted. This is a deliberate trade for safety over raw efficiency.

The cost of this design: forking a process is heavier than spawning a thread, so PostgreSQL historically struggled with very high connection counts (thousands of idle connections waste real OS memory, since each backend has its own process overhead). This is precisely why connection poolers like PgBouncer exist in production: PgBouncer holds a small number of REAL PostgreSQL connections open and multiplexes many client connections onto them, so the database server itself never sees thousands of processes.

Alongside backend processes, several BACKGROUND WORKERS run continuously for the life of the server: the WAL WRITER flushes WAL buffers to disk; the CHECKPOINTER periodically writes all dirty (modified) pages from shared_buffers to disk and records a checkpoint; AUTOVACUUM workers clean up dead row versions (covered in detail this afternoon); and the STATS COLLECTOR aggregates the activity statistics you see in views like pg_stat_activity and pg_stat_user_tables.

Simplify for beginners	Extend for advanced learners
Think of the postmaster as a receptionist at a busy office. Every visitor (client) who arrives gets their OWN dedicated staff member (a backend process) assigned just to them, who handles everything they ask for. If that staff member has a problem, it does not affect anyone else's staff member. The receptionist also has a small permanent team working in the background — cleaners (autovacuum), a security log-keeper (WAL writer), and someone who periodically saves a snapshot of the whole office (checkpointer).	Compare PostgreSQL's process model with MySQL's traditional thread-per-connection model and SQL Server's worker-thread-pool model. Ask: what happens to query plan caching, prepared statement reuse, and CPU cache locality differently in each model? Note that PostgreSQL 18 introduces async I/O improvements specifically to reduce the cost asymmetry of the process model on read-heavy workloads — research this as an optional extension topic if the room has strong systems-engineering background.

DEMO 1.2 — Observing the process model live

```
SELECT pid, username, application_name, state, query
FROM   pg_stat_activity
WHERE  pid = pg_backend_pid();

SELECT pid, username, application_name, state, wait_event_type
FROM   pg_stat_activity
ORDER BY pid;
-- OS process view (run in bash terminal, not psql): ps aux | grep postgres
```

Memory architecture (10 mins)

Memory component	What it does	Default	Production rec. (32GB server)
shared_buffers	Shared cache for recently accessed data pages	128 MB	8 GB (25% of RAM)
work_mem	Per-operation sort/hash memory	4 MB	16–64 MB
maintenance_work_mem	VACUUM, CREATE INDEX memory	64 MB	1 GB
effective_cache_size	Planner hint: total available memory	4 GB	24 GB

Storage architecture (15 mins)

DEMO 1.3 — Storage layout

```
SHOW data_directory;
-- base/ = table/index data, pg_wal/ = WAL segments, global/ = cluster-wide objects
SELECT pg_relation_filepath('pg_class');
SHOW block_size; -- always 8 kB
SELECT relname, relpages, reltuples::BIGINT AS estimated_rows
FROM   pg_class WHERE relname = 'pg_class';
```

Write-Ahead Log (WAL) (35 mins)

THEORY — The write-ahead logging principle and durability (the 'D' in ACID)

The core idea of write-ahead logging is one sentence: NEVER modify a data file on disk until you have first written, and durably flushed, a log record describing that change.

This sounds simple but it is the single mechanism that makes the 'Durability' guarantee in ACID (Atomicity, Consistency, Isolation, Durability) possible at all.

Why log FIRST, before touching the real data? Because writing to the heap files (the actual table data) involves modifying 8kB pages that may be scattered randomly across the disk — random writes are slow and, critically, if the server crashes mid-write, a page can be left PARTIALLY written (a 'torn page'), corrupting the table irrecoverably. The WAL, by contrast, is written SEQUENTIALLY and append-only to files in `pg_wal/` — sequential writes are fast, and each WAL record is checksummed, so a partial write at the very end of the log is detectable and safely ignored on replay.

The recovery protocol works like this: PostgreSQL periodically takes a CHECKPOINT — a guarantee that 'every change up to this WAL position has definitely been written to the actual data files on disk'. If the server crashes, on restart PostgreSQL finds the last checkpoint, then REPLAYS every WAL record written after that checkpoint, in order, re-applying each change to the data files. Because WAL records describe physical changes (not SQL statements), replay is deterministic — re-applying the same WAL record always produces the same result, so recovery is exact, not approximate.

A transaction is only considered COMMITTED once its WAL record has been flushed to disk (an `fsync()` call, bypassing OS write caching) — this is what `synchronous_commit=on` means. Only after that `fsync` returns does PostgreSQL tell the client 'COMMIT successful'. This ordering — log flushed BEFORE acknowledgement — is the entire durability guarantee in one sentence: if the client was told 'yes, committed', the WAL record is provably on durable storage, and recovery can always reconstruct that change even after total power loss.

Simplify for beginners	Extend for advanced learners
Imagine writing down 'I am about to update the diary' on a sticky note BEFORE you actually open the diary and write in it. If the power goes out while you're mid-write in the diary, when it comes back on you can read your sticky notes and finish the job properly. The sticky notes are the WAL. The diary is the real database file.	Discuss the difference between physical (PostgreSQL's default) and logical WAL replication. Physical replay re-applies byte-level page changes — fast, but the standby must be the same PostgreSQL major version and architecture. Logical replication decodes WAL into row-level INSERT/UPDATE/DELETE events, enabling cross-version replication and selective table replication — at higher CPU cost. This distinction matters directly for the NSR's eventual HA topology decisions.

DEMO 1.4 — Observing WAL

```
SELECT pg_current_wal_lsn() AS lsn_before;
INSERT INTO nsr.states (state_code, state_name) VALUES ('XX', 'Test State');
DELETE FROM nsr.states WHERE state_code = 'XX';
SELECT pg_current_wal_lsn() AS lsn_after;
```

```
SELECT name, setting, unit FROM pg_settings
WHERE name IN ('wal_level','synchronous_commit','max_wal_size',
               'checkpoint_completion_target','wal_compression')
ORDER BY name;
```

Multi-Version Concurrency Control (MVCC) (40 mins)

THEORY — MVCC: solving the readers-writers problem without locks

Every database faces the same fundamental concurrency problem: what should happen when one transaction is reading a row while another transaction is simultaneously writing to that same row? Two classical solutions exist.

LOCKING (used by older systems and still used by PostgreSQL for writes-vs-writes): a reader takes a shared lock, a writer takes an exclusive lock, and any conflicting lock request simply waits. This guarantees correctness but creates a serious performance problem — a single long-running report can block every write to the table it reads from, and a single long write can block every read.

MVCC (Multi-Version Concurrency Control), which PostgreSQL implements, takes a fundamentally different approach: instead of locking, NEVER overwrite a row in place. Every UPDATE creates an entirely new physical row version on disk; the old version is not deleted, merely marked as superseded. Each row version is tagged with two hidden system columns: xmin (the ID of the transaction that created this version) and xmax (the ID of the transaction that superseded it — zero if it is still the current version).

When any transaction begins, PostgreSQL determines a SNAPSHOT: a definition of which row versions are 'visible' to it, based on which transaction IDs had committed at the moment the snapshot was taken. A transaction reading data only ever sees row versions whose xmin had committed before its snapshot, and whose xmax either does not exist or had not committed before its snapshot. This is why two transactions running simultaneously can see DIFFERENT, EQUALLY VALID versions of the same row — there is no single 'true' state of the database at any instant, only a set of consistent snapshots.

The price of this design: old row versions accumulate as DEAD TUPLES, consuming disk space and slowing down sequential scans (the scanner must check visibility for every tuple it encounters, even dead ones, until they are physically removed). This is exactly why AUTOVACUUM exists — it is the garbage collector for MVCC, identifying tuples that are no longer visible to ANY active transaction's snapshot and reclaiming that space.

Simplify for beginners	Extend for advanced learners
Imagine a shared Google Doc with 'Track Changes' always on, but instead of showing you the marked-up version, everyone secretly sees a clean copy reflecting only the edits that existed the moment THEY opened the document. Two people can have completely different — but both 'correct' — views of the same paragraph open at the same time, with no need to wait for each	Research PostgreSQL's transaction ID (xid) wraparound problem: xids are 32-bit and wrap around after ~2 billion transactions, which is why VACUUM must eventually 'freeze' old tuples (set a special frozen xmin) to prevent ancient committed data from appearing to be from 'the future' after wraparound. Discuss why this is a real operational risk for a high-throughput NSR

other.

payments table and how to monitor age(datfrozenxid) in pg_database to detect approaching wraparound risk.

DEMO 1.5 — MVCC two-terminal visibility

Instructor note

Open two psql terminals on the projector. Narrate: 'Terminal A is a caseworker in Lagos. Terminal B is a caseworker in Kano updating the same household.'

```
-- TERMINAL A
BEGIN;
SELECT household_id, head_name, pmt_score FROM nsr.households WHERE household_id = 1;

-- TERMINAL B
UPDATE nsr.households SET pmt_score = 99.9 WHERE household_id = 1;

-- TERMINAL A (still in the same transaction)
SELECT household_id, head_name, pmt_score FROM nsr.households WHERE household_id = 1;
-- Under READ COMMITTED: sees B's new value (99.9)
SELECT xmin, xmax, household_id, pmt_score FROM nsr.households WHERE household_id = 1;
COMMIT;
```

Session 4 — Schema development workshop: relational design & normalisation 14:00–17:00

From spreadsheet to relational database (45 mins)

THEORY — The relational model and functional dependency — why normalisation exists

The relational model, formalised by Edgar Codd in 1970, rests on a deceptively simple claim: all data can be represented as RELATIONS (tables) of TUPLES (rows), and any question you might ask of that data can be answered using a small, well-defined set of operations on those relations (selection, projection, join, union — collectively RELATIONAL ALGEBRA, which SQL is a practical implementation of).

Normalisation is the formal discipline of deciding HOW to split your data across tables so that relational algebra works correctly and efficiently. The underlying concept every normal form is built on is FUNCTIONAL DEPENDENCY: column B is functionally dependent on column A if, for any two rows with the same value of A, they must also have the same value of B. For example, state_name is functionally dependent on state_code — given a state_code, the state_name is fully determined.

First Normal Form (1NF) requires that every column hold a single, atomic value (no lists, no repeating groups like payment_jan/payment_feb/payment_mar as separate columns) — this is what makes relational algebra's operations well-defined at all.

Second Normal Form (2NF) requires that every non-key column depend on the WHOLE primary key, not part of it — relevant for composite keys, where a partial dependency (e.g. a column depending only on part of a multi-column key) signals the table should be split.

Third Normal Form (3NF) — the practical target for almost all OLTP schemas, including the NSR — requires that every non-key column depend ONLY on the primary key, and not on another non-key column (no TRANSITIVE dependency). If state_name were stored directly on the households table, household_id determines lga_id determines state_code determines state_name — a transitive chain. 3NF says: pull state_name out into its own table, keyed by state_code, and reference it by foreign key.

Why bother with this formality? Because every functional dependency you fail to respect becomes a place where your data CAN contradict itself. If state_name is duplicated on every household row, nothing in the database prevents 'Kano' on one row and 'KANO' on another for what should be the identical fact. Normalisation does not just organise data neatly — it is the mechanical process of ensuring every fact is stored in exactly one place, so it is structurally impossible for it to disagree with itself.

Simplify for beginners	Extend for advanced learners
Think of normalisation like a well-organised filing cabinet. Instead of writing 'Kano State' on every single form in every drawer (and risking spelling it differently each time), you write a state CODE on each form, and keep one master list, in one drawer, that says what each code means. Change the master list once, and every form automatically means the right thing.	Discuss Boyce-Codd Normal Form (BCNF) — a stricter version of 3NF that handles edge cases involving multiple candidate keys — and denormalisation as a deliberate, informed trade-off: some reporting/analytics tables in a production NSR data warehouse SHOULD be denormalised for read performance, provided the canonical, normalised tables remain the single source of truth and denormalised copies are rebuilt via ETL rather than written to directly.

DEMO 1.6 — The flat data problem

```
\c lab_nsr
CREATE SCHEMA IF NOT EXISTS nsr;
SET search_path = nsr, public;

CREATE TABLE nsr_flat_bad (
  row_id          SERIAL,
  household_code  TEXT,
  head_name      TEXT,
  state          TEXT,    -- 'Kano' / 'KANO' / 'kano state'
  lga            TEXT,
  ward           TEXT,
  num_members    TEXT,    -- '4' or 'four' or '4 people'
  pmt_score      TEXT,    -- '22.5' or 'N/A' or '22,5'
  nin            TEXT,
  payment_jan    TEXT, payment_feb TEXT, payment_mar TEXT
);

INSERT INTO nsr_flat_bad VALUES
(DEFAULT, 'HH-00001', 'Musa Abdullahi', 'Kano', 'Kano Municipal', 'Sharada Ward',
 '4', '22.5', 'AB1234567CD', '5000', '5000', 'failed'),
(DEFAULT, 'HH-00002', 'Aisha Mohammed', 'KANO', 'Kano Municipal', 'Sharada ward',
 '5', '31.2', 'AB1234567CD', '5000', '0', '5000'),
(DEFAULT, 'HH-00001', 'Musa Abdullahi', 'kano state', 'Kano Mun.', 'Sharada Ward',
 '4', '22.5', 'AB1234567CD', '5000', '5000', 'failed');
-- HH-00001 appears TWICE. Same NIN used by two households. 'Kano' spelled three ways.
```


Instructor note

Ask participants to identify ALL problems before revealing the list.

Problems to surface: no PK uniqueness, duplicate household codes, inconsistent state names, NIN repeated across households, TEXT for numbers prevents calculation, columns grow with time.

Live normalisation to 3NF (50 mins)**DEMO 1.7 — Step-by-step normalisation****Instructor note**

Narrate each STEP as you type it — this is where the THEORY above becomes visible.

Step 1 (states): this is the 'master list' from the THEORY box — one row per fact, referenced by code everywhere else. Point out state_name UNIQUE — this is the constraint that makes 'KANO' vs 'Kano' structurally impossible to duplicate.

Step 2 (lgas): demonstrates a FOREIGN KEY referencing Step 1 — this is the mechanical implementation of the functional dependency lga -> state_code.

Step 3 (households, individuals, payments): each FOREIGN KEY you type is a deliberate enforcement of a real-world business rule — pause briefly on each CHECK constraint and ask the room 'what bad data does this specific line prevent?' before moving to the next line. This active-recall pause is more valuable than speed here — it cements the THEORY-to-practice connection for all three audience tiers.

```
-- STEP 1: States reference table
CREATE TABLE nsr.states (
    state_code CHAR(2) PRIMARY KEY,
    state_name TEXT NOT NULL UNIQUE
);

INSERT INTO nsr.states VALUES
('AB','Abia'),('AD','Adamawa'),('AK','Akwa Ibom'),('AN','Anambra'),
('BA','Bauchi'),('BY','Bayelsa'),('BE','Benue'),('BO','Borno'),
('CR','Cross River'),('DE','Delta'),('EB','Ebonyi'),('ED','Edo'),
('EK','Ekiti'),('EN','Enugu'),('GO','Gombe'),('IM','Imo'),
('JI','Jigawa'),('KD','Kaduna'),('KN','Kano'),('KT','Katsina'),
('KE','Kebbi'),('KO','Kogi'),('KW','Kwara'),('LA','Lagos'),
('NA','Nasarawa'),('NI','Niger'),('OG','Ogun'),('ON','Ondo'),
('OS','Osun'),('OY','Oyo'),('PL','Plateau'),('RI','Rivers'),
('SO','Sokoto'),('TA','Taraba'),('YO','Yobe'),('ZA','Zamfara'),
('FC','FCT Abuja');

-- STEP 2: LGAs reference table
CREATE TABLE nsr.lgas (
    lga_id SERIAL PRIMARY KEY,
    lga_name TEXT NOT NULL,
    state_code CHAR(2) NOT NULL REFERENCES nsr.states(state_code),
    UNIQUE (lga_name, state_code)
);

-- STEP 3: Households with proper types and constraints
CREATE TABLE nsr.households (
    household_id BIGSERIAL PRIMARY KEY,
    household_code TEXT NOT NULL UNIQUE,
    head_name TEXT NOT NULL,
    lga_id INT NOT NULL REFERENCES nsr.lgas(lga_id),
    ward TEXT NOT NULL,
    num_members SMALLINT NOT NULL CHECK (num_members BETWEEN 1 AND 50),
    pmt_score NUMERIC(6,3) CHECK (pmt_score >= 0 AND pmt_score <= 100),
    enumerated_at DATE NOT NULL,
```

```

    is_active      BOOLEAN      NOT NULL DEFAULT TRUE
);

CREATE TABLE nsr.individuals (
    individual_id  BIGSERIAL      PRIMARY KEY,
    household_id   BIGINT         NOT NULL REFERENCES nsr.households(household_id)
                        ON DELETE CASCADE,
    nin            CHAR(11)       UNIQUE
                        CHECK (nin IS NULL OR nin ~ '^[A-Z]{2}[0-9]{7}[A-Z]
{2}$'),
    full_name      TEXT          NOT NULL,
    dob            DATE          CHECK (dob > '1900-01-01' AND dob <= CURRENT_DATE),
    gender         CHAR(1)       CHECK (gender IN ('M','F','O'))
);

CREATE TABLE nsr.payments (
    payment_id     BIGSERIAL      PRIMARY KEY,
    household_id   BIGINT         NOT NULL REFERENCES nsr.households(household_id),
    payment_month  DATE          NOT NULL,
    amount_ngn     NUMERIC(12,2) NOT NULL CHECK (amount_ngn > 0),
    payment_channel TEXT          NOT NULL
                        CHECK (payment_channel IN
('bank transfer','mobile money','agent banking','cash')),
    status         TEXT          NOT NULL DEFAULT 'pending'
                        CHECK (status IN ('pending','processing','processed','failed','reversed')),
    processed_at   TIMESTAMPTZ,
    CONSTRAINT uq_household_month UNIQUE (household_id, payment_month)
);

-- Test: try inserting an invalid NIN
INSERT INTO nsr.individuals (household_id, nin, full_name, gender)
VALUES (1, 'AB12345X7CD', 'Test Person', 'M');
-- ERROR: check constraint -- database rejects malformed NIN automatically

```

NSR / Digital Government connection

The states reference table enforces: no 'KANO', 'kano', or 'kano state' can exist.

The NIN regex CHECK means malformed identity numbers are rejected at insert time.

This is data protection by design — a requirement of the Nigeria Data Protection Act 2023.

Entity-relationship model & schema practical (60 mins)

The complete NSR entity model

Entity	Key attributes	Relationships	Cardinality
states	state_code (PK), state_name	Has many LGAs	1:N
lgas	lga_id (PK), lga_name, state_code (FK)	Belongs to state; has many households	N:1, 1:N
households	household_id (PK), household_code, lga_id (FK), pmt_score	Belongs to LGA; has many individuals and payments	N:1, 1:N
individuals	individual_id (PK), household_id (FK), nin, dob	Belongs to household	N:1
payments	payment_id (PK), household_id (FK), payment_month,	Belongs to household	N:1

	amount_ngn		
--	------------	--	--

Instructor note

Time-shortened version of the schema workshop (if the day is running long):

ONE group exercise, not three. All groups design the SAME table — pmt_complaints — then compare designs in a 10-minute plenary rather than three 15-minute presentations.

Full three-group version (if time allows): Group A = pmt_complaints, Group B = address_history, Group C = programme_eligibility. Each: CREATE TABLE with PK, FKs, types, CHECK constraints.

Model answer

-- Model answer: pmt_complaints

```
CREATE TABLE nsr.pmt_complaints (
  complaint_id BIGSERIAL PRIMARY KEY,
  household_id BIGINT NOT NULL REFERENCES nsr.households(household_id),
  filed_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  complaint_type TEXT NOT NULL
    CHECK (complaint_type IN ('score_dispute','missing_payment','data_error','exclusion')),
  description TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'open'
    CHECK (status IN ('open','investigating','resolved','rejected')),
  resolved_at TIMESTAMPTZ,
  resolver_notes TEXT
);
COMMENT ON TABLE nsr.pmt_complaints IS 'Complaints raised by households regarding PMT
scores or payment issues';
```

Practical session — Environment setup & verification (final 30-40 mins of Session 4)

Instructor note

Walk through setup on the projector BEFORE participants start. Pair fast with slow.

Target: every participant running SELECT version(); before close of day — version 18 recommended, but 15/16/17 are equally fine. Mixed versions in the room are normal.

Fast finishers: load the NSR training dataset (see Setup & Resources Guide).

```
psql -U postgres
SELECT version();
-- Expected: PostgreSQL 18.x (or 15.x/16.x/17.x if using an existing install)
CREATE DATABASE lab_nsr; -- if not already created earlier today
\c lab_nsr
CREATE EXTENSION IF NOT EXISTS pgaudit;
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS pgstattuple;
SELECT extname, extversion FROM pg_extension ORDER BY extname;
```

Learning outcome

Participants gain an understanding of modern digital government systems and the role of

PostgreSQL in enterprise-scale data platforms, AND understand how to design scalable, efficient database structures for large datasets.

Intermediate: can connect to PostgreSQL, normalise a flat dataset to 3NF, apply constraints.

Advanced: can explain the process/memory/storage model, WAL, and MVCC precisely, and design self-referential or extension tables with justified ON DELETE behaviour.

WEEK 1
WED 24
JUN

SQL Fundamentals & Advanced Querying

Official theme: SQL Fundamentals & Advanced Querying

Programme objectives addressed: #2 SQL Development & Query Optimisation | #10 Data-Driven Decision Making

Instructor note

Day 3 is the most visible skills day. Participants will send these queries to colleagues.
Progress through the SQL teaching sequence in order — never jump ahead.
The window function demonstrations consistently produce the most engagement.
Every query must answer a real NSR question. Never use abstract examples.

Session 1 — SQL fundamentals: DML and DDL 10:00–11:30

Data Definition Language (DDL) — creating structure (20 mins)

DEMO 3.1 — DDL in context

```
-- DDL creates and modifies database structure
-- Already used: CREATE TABLE, CREATE INDEX

-- ALTER: add a column (non-destructive on large tables)
ALTER TABLE nsr.households
  ADD COLUMN survey_round SMALLINT DEFAULT 1
  CHECK (survey_round BETWEEN 1 AND 10);

-- ALTER: rename a column
ALTER TABLE nsr.households RENAME COLUMN survey_round TO enumeration_round;

-- COMMENT on columns
COMMENT ON COLUMN nsr.households.pmt_score IS
  'Proxy Means Test score 0-100; lower = poorer household';

-- View full table definition including comments
\d+ nsr.households
```

Data Manipulation Language (DML) — managing data (40 mins)

DEMO 3.2 — INSERT, UPDATE, DELETE, SELECT

```
-- INSERT: single row
INSERT INTO nsr.households
  (household_code, head_name, lga_id, ward, num_members, pmt_score, enumerated_at)
VALUES ('HH-TEST-001', 'Fatima Bello', 1, 'Central Ward', 5, 28.750, '2024-03-15');

-- INSERT: from a SELECT (copy qualifying rows to a staging table)
CREATE TABLE nsr.households_review AS
SELECT * FROM nsr.households WHERE pmt_score > 80;

-- UPDATE: correct data
UPDATE nsr.households
```

```

SET      pmt_score = 30.250,
        enumeration_round = 2
WHERE    household_code = 'HH-TEST-001'
RETURNING household_id, household_code, pmt_score;
-- RETURNING shows what was changed without a second SELECT

-- DELETE: with confirmation query first (best practice)
SELECT * FROM nsr.households WHERE household_code = 'HH-TEST-001';
DELETE FROM nsr.households WHERE household_code = 'HH-TEST-001';

-- TRUNCATE: remove all rows quickly (use with caution)
-- TRUNCATE nsr.households_review;
DROP TABLE nsr.households_review;

```

SELECT: the analytical engine (65 mins)

THEORY — SQL is declarative — and the logical order of query processing

SQL is a DECLARATIVE language: you state WHAT result you want, not HOW to compute it. This is fundamentally different from procedural code (Python, Java) where you write the exact steps. In SQL, you write WHAT, and the query planner (covered in depth on Performance Tuning day) decides HOW — choosing join algorithms, scan methods, and execution order on your behalf, guided by table statistics.

This has a surprising and important consequence: the order you TYPE the clauses of a SELECT statement is NOT the order they are logically evaluated. Understanding the real logical order resolves almost every confusing SQL error a learner encounters.

The logical order of evaluation is: FROM (and JOIN) first — determine the working set of rows by combining tables; then WHERE — filter individual rows BEFORE any grouping; then GROUP BY — collapse remaining rows into groups; then HAVING — filter GROUPS (not individual rows) after aggregation; then SELECT — compute the final output columns, including aggregate functions; then DISTINCT; then ORDER BY — sort the final result; then LIMIT/OFFSET — take a slice of the sorted result.

This explains several things that puzzle learners: WHY you cannot reference a column alias defined in SELECT inside the WHERE clause of the same query (WHERE is evaluated before SELECT, so the alias does not exist yet) — but you CAN reference it in ORDER BY (evaluated after SELECT). It also explains WHY WHERE cannot filter on an aggregate function result (COUNT(*), AVG(), etc.) — aggregates do not exist until GROUP BY/SELECT run — which is exactly the gap that HAVING exists to fill.

Simplify for beginners	Extend for advanced learners
You write your shopping list in any order you like — 'milk, then bread, then eggs' — but the shop assistant gathers items in the order that makes sense on the shop floor, not the order on your list. SQL works the same way: you describe what you want, the database decides the most sensible order to actually go and get it.	Tie this back to relational algebra: FROM/JOIN is the algebraic JOIN operator, WHERE is SELECTION (σ), the SELECT clause is PROJECTION (π), and GROUP BY/HAVING correspond to aggregate operators layered on top of the relational core. Ask advanced participants to predict, before running it, what EXPLAIN will show for a query with both WHERE and HAVING filters on the same logical condition expressed

two different ways — and confirm the planner can sometimes push a HAVING condition down to WHERE when it is safe to do so (predicate pushdown).

DEMO 3.3 — SELECT progression

```
-- 1. Basic retrieval
SELECT household_id, household_code, head_name, pmt_score
FROM   nsr.households
LIMIT 10;

-- 2. Filtering
SELECT household_code, head_name, num_members, pmt_score
FROM   nsr.households
WHERE  pmt_score < 33.33
      AND is_active = TRUE
      AND num_members > 5
ORDER BY pmt_score ASC;

-- 3. Aggregate functions
SELECT COUNT(*)                                AS total_households,
       COUNT(*) FILTER (WHERE is_active)       AS active_households,
       ROUND(AVG(pmt_score)::NUMERIC, 2)      AS avg_pmt_score,
       MIN(pmt_score)                         AS min_pmt_score,
       MAX(pmt_score)                         AS max_pmt_score,
       SUM(num_members)                       AS total_registered_persons
FROM   nsr.households;

-- 4. GROUP BY with HAVING
SELECT lga_id,
       COUNT(*)                                AS household_count,
       ROUND(AVG(pmt_score)::NUMERIC, 2)      AS avg_score
FROM   nsr.households
WHERE  is_active = TRUE
GROUP BY lga_id
HAVING COUNT(*) > 100
ORDER BY avg_score ASC;
```

Session 2 — Joins and subqueries 11:45–13:00

JOINS: the most powerful tool in SQL (55 mins)

THEORY — The join as a Cartesian product with a filter — and why three-valued logic matters

Conceptually, every JOIN begins from the CARTESIAN PRODUCT of its input tables: every row of table A paired with every row of table B. A JOIN ... ON condition is then exactly equivalent to: take the Cartesian product, then keep only the pairs that satisfy the ON condition. This is why an accidental JOIN with no ON clause, or with a wrong condition that matches everything, can return millions of rows from two modest tables — you have computed the full Cartesian product by accident.

INNER JOIN keeps only matching pairs — rows from A with no match in B simply vanish from the result entirely. LEFT JOIN (more precisely LEFT OUTER JOIN) keeps every row from the left table regardless of whether a match exists, filling in NULL for every column from the right table when no match is found. This NULL-filling behaviour is exactly what makes LEFT JOIN the correct tool for 'find rows in A that have NO

corresponding row in B' — you LEFT JOIN, then filter WHERE the right-hand key IS NULL.

Here is the subtlety that catches almost everyone at some point: SQL uses THREE-VALUED LOGIC, not the TRUE/FALSE logic of most programming languages. Every comparison involving NULL evaluates to a third value, UNKNOWN, not FALSE. WHERE only keeps rows where the condition evaluates to TRUE — UNKNOWN rows are silently dropped, exactly as if they were FALSE. This is why 'WHERE column != NULL' never matches anything (it is always UNKNOWN, never TRUE) — you must use 'WHERE column IS NOT NULL' instead, which is a special test that bypasses three-valued logic entirely.

This interacts directly with LEFT JOIN: if you LEFT JOIN and then put a condition on the RIGHT table's column inside the WHERE clause (rather than the ON clause), you silently convert your LEFT JOIN back into the equivalent of an INNER JOIN — because any row where the right side is NULL will fail that WHERE condition with UNKNOWN, and get filtered out. The fix is to put right-table filtering conditions in the ON clause, not WHERE, when you genuinely want to keep unmatched left rows.

Simplify for beginners	Extend for advanced learners
Think of two lists: NSR households and NSR payments. An INNER JOIN only keeps households that show up on BOTH lists. A LEFT JOIN keeps every household from the first list no matter what — if a household has no matching payment, you still see them, just with a blank where the payment information would go.	Walk through how PostgreSQL's planner physically executes a JOIN using one of three algorithms — Nested Loop, Hash Join, or Merge Join — and what statistics (table size, index availability, sortedness) drive the planner's choice between them. This is the natural bridge into tomorrow's EXPLAIN ANALYZE work on the Performance Tuning day: ask advanced participants to predict which join algorithm PostgreSQL will choose for the 'households never paid' query before running EXPLAIN to check.

DEMO 3.4 — JOIN types with NSR data

```
-- INNER JOIN: only households that have a matching LGA
SELECT h.household_code, h.head_name,
       l.lga_name, s.state_name, h.pmt_score
FROM   nsr.households h
JOIN   nsr.lgas      l ON l.lga_id      = h.lga_id
JOIN   nsr.states    s ON s.state_code = l.state_code
WHERE  h.is_active = TRUE
ORDER  BY s.state_name, h.pmt_score ASC
LIMIT 50;

-- LEFT JOIN: ALL active households + their last payment (NULL if none)
SELECT h.household_code, h.head_name, h.pmt_score,
       MAX(p.payment_month) AS last_payment_month,
       MAX(p.status)        AS last_payment_status
FROM   nsr.households h
LEFT   JOIN nsr.payments p
       ON p.household_id = h.household_id
       AND p.status = 'processed'
WHERE  h.is_active = TRUE
GROUP  BY h.household_code, h.head_name, h.pmt_score;

-- The critical query: households that NEVER received a payment
SELECT h.household_code, h.head_name, h.pmt_score,
```



```

s.state_name, l.lga_name
FROM nsr.households h
JOIN nsr.lgas l ON l.lga_id = h.lga_id
JOIN nsr.states s ON s.state_code = l.state_code
LEFT JOIN nsr.payments p ON p.household_id = h.household_id
WHERE p.payment_id IS NULL -- LEFT JOIN found no match = no payments ever
AND h.is_active = TRUE
ORDER BY h.pmt_score ASC; -- poorest excluded households first

```

NSR / Digital Government connection

The 'never received a payment' query is one of the most operationally important queries the NSR team can run. It identifies programme exclusion errors — households that are registered, eligible, and active, but somehow not receiving their entitlement. This query can be run monthly as part of programme quality assurance.

Subqueries (25 mins)

DEMO 3.5 — Subquery patterns

```

-- Scalar subquery: find households with above-average pmt_score
SELECT household_code, head_name, pmt_score
FROM nsr.households
WHERE pmt_score > (SELECT AVG(pmt_score) FROM nsr.households WHERE is_active = TRUE)
AND is_active = TRUE
ORDER BY pmt_score DESC;

-- IN subquery: households in states beginning with 'K'
SELECT household_code, head_name
FROM nsr.households
WHERE lga_id IN (
  SELECT l.lga_id
  FROM nsr.lgas l
  JOIN nsr.states s ON s.state_code = l.state_code
  WHERE s.state_name LIKE 'K%'
)
AND is_active = TRUE;

-- EXISTS subquery: households with at least one NIN-verified member
SELECT h.household_code, h.head_name
FROM nsr.households h
WHERE EXISTS (
  SELECT 1 FROM nsr.individuals i
  WHERE i.household_id = h.household_id
  AND i.nin IS NOT NULL
  AND i.nin_verified = TRUE
)
AND h.is_active = TRUE;

```

Session 3 — Window functions 14:00–15:30

Window functions: analytics without collapsing rows (65 mins)

THEORY — The window frame: a sliding context attached to each row

GROUP BY answers 'what is true of this GROUP of rows?' by collapsing many rows into one summary row per group — you lose access to the individual rows once you aggregate. Window functions answer a different question: 'what is true of the rows AROUND this particular row?' — and crucially, they do this WITHOUT collapsing anything. Every original row remains in the output, with an extra computed column attached.

The mechanism has three logical parts. **PARTITION BY** divides the full result set into independent groups — exactly like **GROUP BY** conceptually, except no collapsing happens; think of it as 'restart the calculation at the start of each new partition'. **ORDER BY** (inside the **OVER** clause) defines a sequence within each partition — this is what makes **RANK()**, **LAG()**, and running totals meaningful, since 'previous row' and 'rank' only make sense relative to some order. The **FRAME** clause (**ROWS BETWEEN ... AND ...**) defines exactly which neighbouring rows, within the ordered partition, are visible to the calculation for **THIS** row — by default, for most aggregate window functions, the frame is 'everything from the start of the partition up to and including the current row', which is precisely what makes **SUM() OVER (ORDER BY ...)** produce a running total rather than a single grand total.

This is why **RANK()** and **DENSE_RANK()** differ subtly: **RANK()** leaves gaps after ties (1, 2, 2, 4 — rank 3 is skipped because two rows tied for 2nd) because it counts how many rows strictly precede the current row in the ordering, plus one. **DENSE_RANK()** instead counts how many **DISTINCT** preceding values exist, so ties never create gaps (1, 2, 2, 3). Neither is 'more correct' — the right choice depends on whether your downstream logic needs to know exactly how many households share an identical PMT score (use **RANK**, gaps included) or just needs a compact priority tier (**DENSE_RANK**).

Simplify for beginners	Extend for advanced learners
Imagine standing in a queue and being told your position number, but everyone can still see the whole queue — nobody actually leaves. That's a window function: everyone (every row) gets an extra label (their rank, their running total) without anyone being removed from view, unlike GROUP BY where people get merged into teams and you can only see the team, not the individuals.	Have advanced participants research the difference between RANGE and ROWS frame modes (ROWS BETWEEN counts physical rows; RANGE BETWEEN counts logical value differences, relevant for handling ties or date-based windows like 'the last 7 calendar days' rather than 'the last 7 rows'). This distinction is essential for correctly implementing rolling NSR payment-compliance windows when payment dates are not perfectly regular across households.

DEMO 3.6 — Poverty ranking and quintile analysis

```
-- Rank households within each state by PMT score (lowest = rank 1 = poorest)
SELECT h.household_code,
       h.head_name,
       s.state_name,
       h.pmt_score,
       RANK() OVER (
         PARTITION BY l.state_code
         ORDER BY h.pmt_score ASC
       ) AS poverty_rank_in_state,
       NTILE(5) OVER (ORDER BY h.pmt_score ASC) AS national_quintile,
       ROUND(
         PERCENT_RANK() OVER (ORDER BY h.pmt_score ASC) * 100,
         1) AS national_percentile
FROM   nsr.households h
JOIN   nsr.lgas l ON l.lga_id = h.lga_id
JOIN   nsr.states s ON s.state_code = l.state_code
WHERE  h.is_active = TRUE;

-- Month-over-month payment compliance per state using LAG
```

```

WITH monthly AS (
  SELECT s.state_code, s.state_name,
         DATE_TRUNC('month', p.payment_month)::DATE AS month,
         ROUND(100.0 * COUNT(*) FILTER (WHERE p.status='processed')
              / NULLIF(COUNT(*),0),1) AS compliance_pct
  FROM   nsr.payments p
  JOIN   nsr.households h ON h.household_id = p.household_id
  JOIN   nsr.lgas l ON l.lga_id = h.lga_id
  JOIN   nsr.states s ON s.state_code = l.state_code
  GROUP BY s.state_code, s.state_name,
           DATE_TRUNC('month', p.payment_month)
)
SELECT state_name, month, compliance_pct,
       LAG(compliance_pct) OVER (
         PARTITION BY state_code ORDER BY month
       ) AS prev_month_pct,
       compliance_pct - LAG(compliance_pct) OVER (
         PARTITION BY state_code ORDER BY month
       ) AS change
FROM   monthly
ORDER BY state_name, month;

```

Session 4 — Practical laboratory 15:45–17:00

Lab exercises

Instructor note

Exercises are in the Participant Workbook Day 3 section.

Circulate continuously. Fast finishers: the challenge query at the end.

Debrief: compare solutions across the room — multiple approaches are valid.

Model answers for key exercises

Model answer

-- Q: State-level NIN coverage report

```

SELECT s.state_name,
       COUNT(i.individual_id)           AS total_individuals,
       COUNT(i.nin)                     AS with_nin,
       COUNT(*) - COUNT(i.nin)          AS missing_nin,
       ROUND(100.0 * COUNT(i.nin) / NULLIF(COUNT(*),0),1) AS nin_coverage_pct
FROM   nsr.states s
JOIN   nsr.lgas l ON l.state_code = s.state_code
JOIN   nsr.households h ON h.lga_id = l.lga_id AND h.is_active = TRUE
JOIN   nsr.individuals i ON i.household_id = h.household_id
GROUP BY s.state_name ORDER BY nin_coverage_pct ASC;

```

Learning outcome

Participants acquire practical SQL skills for querying, managing and analysing large datasets.

Intermediate: can write SELECT with JOINS, GROUP BY, HAVING, and subqueries.

Advanced: can write window functions for ranking and trend analysis, optimise query structure.

WEEK 1
THU 25
JUN

PostgreSQL Administration, Security & Backup Management

Official theme: PostgreSQL Administration, Security & Backup Management

Programme objectives addressed: #1 PostgreSQL Database Administration | #6 Data Governance | #7 Database Security

Important

SCHEDULE CHANGE — this content has moved from Friday to Thursday.

The Innovation Hub, Birmingham study visit (originally Thursday) has moved to Friday 26 June to make room for the technical sequence opening on Tuesday 23 June. No content has been cut from this session — it runs exactly as originally planned, just one day earlier.

Instructor note

Security resonates strongly with senior officials. Connect to the NDP Act 2023 throughout. The RBAC and RLS demonstrations are the technical highlight of the week for policy participants. Backup and disaster recovery is the most practical section — everyone has experienced data loss. End with the Week 1 reflection: returning to the Day 1 flipchart is a powerful closing moment before tomorrow's study visit to Innovation Hub, Birmingham.

Session 1 — User, role management & RBAC 10:00–11:30

PostgreSQL security model overview (25 mins)

THEORY — Defence in depth, and RBAC as a formal access-control model

Security engineering has a foundational principle called DEFENCE IN DEPTH: no single control should be the only thing standing between an attacker (or an honest mistake) and your data. PostgreSQL implements this as three independent layers, each of which must be satisfied before any data is returned.

Layer 1 — CONNECTION: governed by `pg_hba.conf` (host-based authentication), this decides WHICH client addresses, on WHICH network, are even allowed to attempt to connect, and over what protocol (plain TCP or TLS-encrypted). A request that fails this layer never reaches authentication at all.

Layer 2 — AUTHENTICATION: having been allowed to connect, the client must prove WHO they are — typically via SCRAM-SHA-256 password authentication (a modern, salted, challenge-response protocol that never sends the password itself over the wire), or certificate-based authentication, or external identity providers (LDAP, and as of PostgreSQL 18, OAuth 2.0).

Layer 3 — AUTHORISATION: having proven identity, what is this specific identity permitted to DO? This is where Role-Based Access Control (RBAC) lives. RBAC is a formal model, not a PostgreSQL invention — it separates PRINCIPALS (who is asking), ROLES (a named bundle of permissions), and PRIVILEGES (specific permitted actions on specific objects). Crucially, principals are assigned to roles, and roles are granted privileges — principals are never granted privileges directly in a well-designed system. This indirection is what makes RBAC manageable at scale: when a caseworker's job changes, you change their ROLE MEMBERSHIP (one operation), rather than auditing and rewriting every individual GRANT statement that ever touched their account.

PostgreSQL's specific implementation lets roles inherit from other roles, forming a directed graph of privilege — exactly as we build today, where `nsr_supervisor` inherits everything `nsr_caseworker` can do, then adds more. This hierarchical inheritance directly implements the security principle of LEAST PRIVILEGE: every role should hold the minimum set of privileges required for its function, and nothing more — precisely what Article 2.6 of the Nigeria Data Protection Act 2023 requires in legal language.

Simplify for beginners	Extend for advanced learners
Think of role-based access like staff ID badges in a building. Instead of giving every individual employee a custom, hand-written set of door keys, you give them a badge type: 'Caseworker', 'Supervisor', 'Security'. Each badge type opens exactly the doors that job needs. When someone gets promoted, you swap their badge type — you don't need to reprogram every single door.	Discuss the distinction between RBAC and Attribute-Based Access Control (ABAC) — ABAC makes access decisions based on dynamic attributes (time of day, request context, data sensitivity classification) rather than fixed role membership. The Row-Level Security policy built later today, which reads a session variable (<code>app.state_code</code>) at query time, is actually a hybrid: RBAC determines WHICH table operations a role may attempt, while the RLS policy itself behaves more like ABAC, evaluating a runtime attribute to decide WHICH ROWS are visible.

DEMO 4.1 — The NSR role hierarchy

```
-- Create group roles (NOLOGIN: these are privilege containers, not login accounts)
CREATE ROLE nsr_viewer NOLOGIN;
CREATE ROLE nsr_caseworker NOLOGIN;
CREATE ROLE nsr_supervisor NOLOGIN;
CREATE ROLE nsr_dba NOLOGIN;
CREATE ROLE nsr_auditor NOLOGIN;

-- Role inheritance: supervisor inherits caseworker privileges
GRANT nsr_caseworker TO nsr_supervisor;

-- Privilege grants
GRANT USAGE ON SCHEMA nsr TO nsr_caseworker, nsr_viewer, nsr_auditor;

-- Caseworker: read household and individual data only
GRANT SELECT ON nsr.households, nsr.individuals, nsr.lgas, nsr.states
TO nsr_caseworker;

-- Supervisor: caseworker privileges + manage payments
GRANT SELECT, UPDATE ON nsr.payments TO nsr_supervisor;
```

```

GRANT SELECT, INSERT ON nsr.households TO nsr_supervisor;

-- DBA: full access
GRANT ALL ON ALL TABLES IN SCHEMA nsr TO nsr_dba;
GRANT ALL ON ALL SEQUENCES IN SCHEMA nsr TO nsr_dba;

-- Auditor: read everything; every query will be logged
GRANT SELECT ON ALL TABLES IN SCHEMA nsr TO nsr_auditor;

-- Create actual login users
CREATE USER kano_worker WITH PASSWORD 'NSR2026Kano!' IN ROLE nsr_caseworker;
CREATE USER lagos_super WITH PASSWORD 'NSR2026Lagos!' IN ROLE nsr_supervisor;
CREATE USER ministry_view WITH PASSWORD 'NSR2026Min!' IN ROLE nsr_viewer;
CREATE USER wb_auditor WITH PASSWORD 'NSR2026Aud!' IN ROLE nsr_auditor;

-- Verify the role tree
SELECT r.rolname, STRING_AGG(m.rolname, ', ') AS member_of
FROM   pg_roles r
LEFT   JOIN pg_auth_members am ON am.member = r.oid
LEFT   JOIN pg_roles m ON m.oid = am.roleid
WHERE  r.rolname LIKE 'nsr_%' OR r.rolname IN ('kano_worker','lagos_super')
GROUP  BY r.rolname ORDER BY r.rolname;

```

Row-Level Security (RLS) (45 mins)

THEORY — RLS as an invisible, mandatory WHERE clause enforced by the database engine

Standard PostgreSQL privileges (GRANT SELECT, etc.) are TABLE-level: a role either can or cannot read a table, with no finer granularity. This is a problem the moment you need 'a Lagos caseworker may read the households table, but only the rows belonging to Lagos'. Without a database-level mechanism for this, that restriction has to live in application code — and application code can have bugs, can be bypassed by a direct database connection, and must be re-implemented correctly in every single application that ever touches the database.

Row-Level Security (RLS), standardised in the SQL standard and implemented in PostgreSQL since version 9.5, solves this by moving the restriction INTO the database engine itself, where it cannot be bypassed by any client, regardless of how that client connects. When RLS is enabled on a table and a POLICY is defined, PostgreSQL transparently rewrites every query against that table to include an additional, mandatory filtering condition — conceptually appending 'AND (policy_condition)' to the WHERE clause of every SELECT, UPDATE, or DELETE, even though the client never wrote that condition themselves and cannot remove it.

This is precisely why we use a SESSION-level variable — `current_setting('app.state_code')` — rather than hard-coding a value into each policy. The policy's condition is evaluated FRESH on every query, reading whatever value the current session has set, which lets a single static policy definition correctly restrict thousands of different caseworkers to thousands of different states, with the application simply responsible for setting the correct session variable when a user authenticates — and even if the application gets that step wrong, the WORST outcome is access denied (an empty result), never accidental over-exposure, because of how the NULLIF pattern is constructed in today's policy.

RLS composes correctly with RBAC: GRANT determines whether a role may touch a table

AT ALL; RLS then determines, for roles that may touch it, WHICH ROWS they see. Roles marked BYPASSRLS (such as nsr_dba and nsr_supervisor in today's design) skip row filtering entirely — a deliberate, audited exception, not a bypass of the model.

Simplify for beginners	Extend for advanced learners
Imagine a filing cabinet where every drawer has an invisible rule taped inside it: 'Only show files matching the visitor's ID badge'. Nobody opening the drawer can see files that don't match their badge, even if they ask for everything — the cabinet itself quietly only shows them what they're allowed to see, no matter how they ask.	Discuss the performance implications of RLS: because the policy condition is logically ANDed into the query, the planner can sometimes use it to drive index selection (as we will verify with EXPLAIN), but a poorly written policy — for instance, one that calls an expensive function per row rather than a simple indexed comparison — can silently force a sequential scan across an entire large table. Ask advanced participants to predict, then verify, whether today's policy uses the lga_id index efficiently or forces extra computation per row.

DEMO 4.2 — State-level RLS

```
-- Enable RLS on the households table
ALTER TABLE nsr.households ENABLE ROW LEVEL SECURITY;

-- Policy: caseworkers see only their assigned state
-- The application sets app.state_code at session start
CREATE POLICY caseworker_state_policy
ON nsr.households
FOR ALL
TO nsr_caseworker
USING (
    lga_id IN (
        SELECT l.lga_id FROM nsr.lgas l
        WHERE l.state_code = NULLIF(
            current_setting('app.state_code', TRUE), ''
        )
    )
);

-- Supervisors bypass RLS (see all states)
ALTER ROLE nsr_supervisor BYPASSRLS;
ALTER ROLE nsr_dba        BYPASSRLS;

-- Test: connect as caseworker with Kano state
SET ROLE nsr_caseworker;
SET app.state_code = 'KN';
SELECT COUNT(*) FROM nsr.households;
-- Returns ONLY Kano households

-- What if state_code is not set? (NULLIF prevents showing all rows)
RESET app.state_code;
SELECT COUNT(*) FROM nsr.households;
-- Returns 0 rows -- not all rows. Safer failure mode.

RESET ROLE;
```


Session 2 — Security best practices & pgAudit 11:45–13:00

pgAudit: compliance-grade audit logging (40 mins)

DEMO 4.3 — Configuring pgAudit

```
-- Verify pgAudit is loaded
SELECT name, setting FROM pg_settings WHERE name = 'shared_preload_libraries';
CREATE EXTENSION IF NOT EXISTS pgaudit;

-- Configure session-level audit per role
ALTER ROLE nsr_auditor SET pgaudit.log = 'read';           -- every SELECT logged
ALTER ROLE nsr_dba     SET pgaudit.log = 'all';           -- all operations logged

-- Database-level: log all writes and DDL
ALTER DATABASE lab_nsr SET pgaudit.log = 'write,ddl';
ALTER DATABASE lab_nsr SET pgaudit.log_relation = on;
ALTER DATABASE lab_nsr SET pgaudit.log_catalog = off;
SELECT pg_reload_conf();

-- Application-layer audit table (queryable, structured)
CREATE TABLE nsr.audit_log (
    log_id      BIGSERIAL PRIMARY KEY,
    event_time  TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp(),
    db_user     TEXT        NOT NULL DEFAULT session_user,
    operation   TEXT        NOT NULL,
    table_name  TEXT        NOT NULL,
    record_id   BIGINT,
    old_data    JSONB,
    new_data    JSONB
);

-- Prevent modification of audit log
REVOKE DELETE, UPDATE ON nsr.audit_log FROM PUBLIC;

-- Trigger to capture all changes to households
CREATE OR REPLACE FUNCTION nsr.fn_audit()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER AS $$
BEGIN
    INSERT INTO nsr.audit_log(operation, table_name, record_id, old_data, new_data)
    VALUES (TG_OP, TG_TABLE_NAME,
            CASE WHEN TG_OP='DELETE' THEN OLD.household_id ELSE NEW.household_id END,
            CASE WHEN TG_OP IN ('UPDATE', 'DELETE') THEN to_jsonb(OLD) END,
            CASE WHEN TG_OP IN ('INSERT', 'UPDATE') THEN to_jsonb(NEW) END);
    RETURN COALESCE(NEW, OLD);
END; $$;

CREATE TRIGGER trg_audit_households
AFTER INSERT OR UPDATE OR DELETE ON nsr.households
FOR EACH ROW EXECUTE FUNCTION nsr.fn_audit();
```

Session 3 — Backup, recovery & disaster recovery 14:00–15:30

Backup strategies (55 mins)

THEORY — RPO and RTO: the two numbers that should drive every backup decision

Every backup strategy decision should start from two numbers, not from a tool preference. RECOVERY POINT OBJECTIVE (RPO) answers: 'how much data can we afford to lose?' — expressed as a time duration. An RPO of 1 hour means that in the worst case, after a disaster, you accept losing up to 1 hour of the most recent transactions. RECOVERY TIME OBJECTIVE (RTO) answers a different question: 'how long can we afford to be DOWN?' — the time from disaster to the system being usable

again. These two numbers are independent and both matter: a backup strategy with an excellent RPO but a terrible RTO means you lose almost no data, but customers — or in the NSR's case, beneficiaries waiting on payments — cannot be served for hours.

Logical backups (`pg_dump`) capture the LOGICAL content of the database — the SQL statements needed to recreate it — independent of PostgreSQL's internal file format. This makes them portable across PostgreSQL versions and operating systems, but restoring requires literally re-running all those statements and rebuilding every index from scratch, which is slow on large databases — a poor RTO at scale.

Physical backups (`pg_basebackup`) copy the actual data files. Restoring is just copying files back and starting PostgreSQL — much faster (better RTO) — but the backup is only as fresh as when it was taken, so without WAL archiving alongside it, your RPO is bounded by your backup frequency (e.g. 'backup nightly' = up to 24 hours of potential data loss).

POINT-IN-TIME RECOVERY (PITR) combines a physical base backup with continuous WAL archiving (every WAL segment, as it is generated, copied to separate storage). To recover, you restore the base backup, then REPLAY archived WAL up to any chosen moment — including 'one second before someone ran that accidental DROP TABLE'. This is what gives continuous archiving its near-zero RPO: you can recover to almost any point in time, not just to whenever the last full backup happened to run.

STREAMING REPLICATION takes this further: a standby server continuously receives WAL over the network in near real-time and replays it immediately, keeping a near-live copy ready to be promoted to primary within seconds — this is the only approach offering RTO measured in seconds rather than minutes or hours, which is why it is the correct target architecture for an NSR production system serving millions of beneficiaries who cannot tolerate extended outages.

Simplify for beginners	Extend for advanced learners
RPO is 'how much work are you willing to redo?' and RTO is 'how long can the office be closed?' A photocopy of all your paperwork from last night (logical/physical backup) might mean redoing a day's work. A live mirror office running in another building, always up to date (streaming replication), means almost no redone work and almost no closure at all — but it costs much more to maintain.	Have advanced participants calculate the actual RPO/RTO trade-off for a specific NSR scenario: a corrupted households table discovered 6 hours after the corrupting UPDATE ran. Walk through exactly which WAL position to recover to using PITR, and discuss why 'recover to immediately before the bad transaction's commit' requires knowing the precise transaction ID or timestamp — and how <code>pg_waldump</code> could be used to identify it forensically from the archived WAL stream itself.

Method	Tool	Use case	RPO	RTO	NSR suitability
Logical backup	<code>pg_dump</code>	Selective table backups, migrations	Hours	Hours	Development & migration

Physical backup	pg_basebacku p	Full server backup for recovery	Minut es	Minut es	Production staging backup
Continuous archiving	pgBackRest/ Barman	Point-in-time recovery	Near zero	Minut es	Production primary database
Streaming replication	Built-in	Hot standby for HA and read scaling	Near zero	Secon ds	NSR production HA requirement

DEMO 4.4 — pg_dump and pg_restore

```
-- LOGICAL BACKUP: pg_dump
-- Dumps the nsr schema to a custom-format file (compressed, parallel-restoreable)
-- Run in bash (not psql):
-- pg_dump -U postgres -d lab_nsr -n nsr -F c -f /tmp/nsr_backup.dump

-- List contents of the backup
-- pg_restore -l /tmp/nsr_backup.dump

-- Restore to a new database
-- createdb -U postgres lab_nsr_restore
-- pg_restore -U postgres -d lab_nsr_restore -F c /tmp/nsr_backup.dump

-- SQL format: human-readable, useful for inspecting
-- pg_dump -U postgres -d lab_nsr -n nsr -F p -f /tmp/nsr_backup.sql

-- Verify backup integrity
-- pg_restore --list /tmp/nsr_backup.dump | wc -l

-- From inside psql: see current WAL archiving status
SELECT * FROM pg_stat_archiver;
```

DEMO 4.5 — pg_basebackup for physical backup

```
-- Physical backup: copies the entire data directory
-- Run in bash:
-- pg_basebackup -U postgres -D /tmp/nsr_physical_backup
-- -Ft -z -P --wal-method=stream

-- -Ft: tar format
-- -z: compress with gzip
-- -P: show progress
-- --wal-method=stream: include WAL in backup for immediate use

-- Check backup exists
-- ls -lh /tmp/nsr_physical_backup/

-- PostgreSQL configuration for WAL archiving
SELECT name, setting FROM pg_settings
WHERE name IN ('archive_mode', 'archive_command', 'archive_status')
ORDER BY name;
```

Disaster recovery planning (25 mins)

Scenario	Impact	Detection	Recovery procedure	Target RTO
Single disk failure	Data file corruption	PostgreSQL error on access	Restore from physical backup + replay WAL	< 2 hours

Server failure (power/hardware)	All connections lost	Monitoring alert	Promote standby replica (if configured)	< 5 minutes
Accidental DELETE	Data loss	Application error report	PITR to timestamp before DELETE	< 30 minutes
Ransomware / full server	Complete loss	Detection by IT security	Restore from offsite backup	< 4 hours

Session 4 — Backup simulation lab 15:45–17:00

Instructor note

Participants run `pg_dump` and `pg_restore` in pairs on their training instances.
 The simulation exercise: each participant drops a table, then restores it from their backup.
 This is the most memorable practical of the week — doing a recovery is visceral.
 Fast finishers: configure WAL archiving and verify `archive_status` shows 'ok'.

Learning outcome

Participants develop practical administration skills necessary for managing enterprise PostgreSQL environments.
 Intermediate: can perform `pg_dump/restore`, understand the role model, and configure basic `pgAudit`.
 Advanced: can configure RLS with session variables, explain PITR recovery, design a DR plan with RTO/RPO targets.

WEEK 1
FRI 26 JUN**Study Visit 1 — Innovation Hub, Birmingham****Official theme:** Local Government Digital Transformation & Citizen Services**Programme objectives addressed:** #8 Digital Welfare Administration | #9 Social Protection Systems | #12 Public Sector Digital Transformation**Important**

SCHEDULE CHANGE — this visit has moved from Thursday to Friday.
 Friday was originally a free day; it now hosts the Innovation Hub, Birmingham visit so the technical sequence (Tue/Wed/Thu) can run uninterrupted. No content has changed — only the date.

Instructor brief

You are a guest in Innovation Hub, Birmingham environment. Brief participants firmly on conduct before departure — this is the first external visit of the programme.

Your role during the visit: facilitator and bridge-builder, not lecturer.

After the visit: the debrief session is your most important facilitation of the day.

Suggested debrief structure: What? (what did you observe) / So what? (what does it mean) / Now what? (how does it apply to the NSR)

Before departure — final briefing (15 mins)

- Assign observation roles (Technology / Governance / Process / Policy) — see the Participant Workbook observation matrix
- Remind: active listening, specific questions, note-taking required
- Technology focus: ask about their database stack, citizen ID strategy, data sharing protocols
- Governance focus: ask how they handle Subject Access Requests and data breach response

Focus areas during the visit

Focus area	What to observe	Question to ask if the opportunity arises
Digital service delivery	How citizens access services online vs in person	What percentage of services are now fully digital, and what's the barrier for the rest?
Citizen data management	How records are created, maintained, deduplicated	How do you identify the same citizen across different council departments?
Community welfare services	How vulnerable households are identified and supported	How do you find people who need help before they ask for it?
Data governance	GDPR compliance, data sharing agreements	How do you handle a Subject Access Request end to end?

Digital inclusion	How non-digital citizens are served	How does this compare with serving NSR beneficiaries with no internet access?
-------------------	-------------------------------------	---

Evening debrief (allow 90 minutes)

Debrief framework — What / So What / Now What

WHAT did you observe? (15 mins — open contributions from the room)

SO WHAT does it mean? (30 mins — structured discussion by focus area)

Technology: what PostgreSQL or equivalent concepts did you see in use?

Governance: what data protection practices map to the NSR context?

Process: what citizen service workflows could Nigeria adapt?

NOW WHAT? (30 mins — each participant writes 3 applicable lessons in their workbook)

Preview the Week 2 sequence: Mid-Training Assessment (Mon), Performance Tuning (Tue), ETL/ELT & Governance (Wed).

Instructor background notes (do not distribute to participants)

Innovation Hub, Birmingham — Context

The Innovation Hub, Birmingham is a flagship enterprise and technology centre in the West Midlands, supporting digital innovation and business growth.

Digital infrastructure: cloud-first, Azure-hosted services supporting data analytics, start-up incubation, and enterprise operations.

Data management: shared platform for innovation projects, tenant organisations, and hosted digital services.

Relevant parallels: universal services, means-tested support, digital inclusion challenges.

Data governance: ICO registered; UK GDPR compliance; data sharing agreements with DWP and NHS.

Questions that tend to generate the richest responses:

'How do you identify households that need multiple types of support simultaneously?'

'How do you handle data quality when records come from multiple departments?'

Learning outcome

Participants gain insights into how local government organisations use data and digital systems to improve citizen services and community outcomes.

All participants: can identify at least three transferable practices for the NSR context.

WEEK 2
MON 29
JUN

Mid-Training Assessment Day

Official theme: Self-directed practical assessment — consolidating Tue 23 / Wed 24 / Thu 25 June content

Format: Practical SQL/PostgreSQL tasks only. No written or reflection component.

Important

SCHEDULE CHANGE — this day did not exist in the original programme.

It replaces what was originally 'Performance Tuning & Database Optimisation' (now moved to Tuesday 30 June). The Lead Instructor is NOT present in the morning and joins at 14:00.

A non-technical proctor supervises the room from 10:00 to 14:00.

This page and the following pages must be read by the proctor before the day begins — share the 'Proctor instructions' box below with them in advance, ideally Friday 26 June.

Purpose of this day

This is a checkpoint, not a new teaching day. Participants work independently through a fixed set of practical tasks covering everything taught on Tuesday (Foundations + Architecture), Wednesday (SQL Fundamentals & Advanced Querying), and Thursday (Administration, Security & Backup). The results tell you, before Week 2's remaining content, exactly where the group's gaps are.

Proctor instructions — read this before Monday

Who the proctor is and what they need to do

The proctor does NOT need any PostgreSQL or SQL knowledge. Their job is logistics and integrity, not technical support.

Their responsibilities for 10:00–14:00:

1. Open the room at 09:30. Take attendance when participants are settled. Training starts at 10:00. Distribute the Participant Workbook assessment pages at 10:00.
2. Read the timing instructions aloud at 10:00 (provided below) — word for word is fine.
3. Keep time. Announce when 50% of the time has elapsed, and again 15 minutes before the end.
4. Walk the room periodically. You are checking that people are working individually on their own laptops, not checking whether their SQL is correct — you do not need to understand the answers to do this.
5. If a participant asks a technical question ('is this SQL right?', 'why am I getting an error?'), respond with: 'I can't help with the technical content today — please note the question and the lead instructor will go through it from 2pm.' Do not guess.
6. Collect or have participants save their completed workbook pages by 13:00.
7. At 13:00, dismiss for lunch. Tell the room the lead instructor joins at 14:00 to review.

Escalation: if a laptop or PostgreSQL installation fails entirely and a participant cannot proceed, note their name and the issue, let them continue on paper/pseudocode where possible, and flag it for the lead instructor at 14:00. Do not attempt to fix it yourself.

Morning — independent assessment (10:00–13:00, proctor only)

Time	Session
09:30–10:00	Arrival, attendance, workbook distribution. Proctor reads timing instructions aloud.
10:00–11:30	Part A — Foundations & Architecture tasks (from Tuesday 23 June content)
11:30–11:45	Break
11:45–13:00	Part B — SQL Fundamentals & Advanced Querying tasks (from Wednesday 24 June content)
13:00 (straight to lunch)	Part C — Administration, Security & Backup tasks (from Thursday 25 June content)
13:00–14:00	Lunch

Script for the proctor to read aloud at 10:00

Read word-for-word

'Good morning. Today is a self-directed assessment day. You have three parts to complete, labelled A, B, and C in your workbook, covering Tuesday, Wednesday, and Thursday's content. Work through each part using your own PostgreSQL installation and the lab_nsr database. Each part has a suggested time limit — these are guides, not hard cut-offs, but please try to keep pace so you finish Part C before lunch.

This is an individual assessment — please work on your own rather than with a neighbour. I am here to support the room logistically, but I am not able to answer technical questions about SQL or PostgreSQL today. If you get stuck, note down where and why, and write your best attempt — partial answers are valuable and will be reviewed this afternoon.

Dr Anande joins us at 2pm to go through everything in detail.

Please save your work as you go. Let's begin.'

Marking rubric — for the Lead Instructor's afternoon review

Part	Task focus	Pass indicator	Common gap to watch for
A — Foundations & Architecture	Connect to psql; explain process model; identify a WAL/MVCC scenario	Correct connection; can explain in own words why WAL prevents data loss	Confusing WAL with backup; vague process-model explanation

B — SQL Fundamentals & Advanced Querying	Write a JOIN-based report; use at least one window function	Query executes and returns correct NSR-relevant result	JOIN direction errors (INNER vs LEFT); incorrect window frame
C — Administration, Security & Backup	Create a role with restricted privileges; describe a backup approach	Role created with correct GRANT scope; backup method matches the scenario	Granting overly broad privileges (ALL instead of SELECT); confusing logical vs physical backup

Tip

Use this rubric to triage, not grade formally. The goal is to identify which 2-3 topics need a 10-minute refresher woven into Tuesday/Wednesday's remaining content, not to assign marks. A simple 'solid / shaky / gap' per participant per part is enough to plan the afternoon.

Afternoon — Lead Instructor joins 14:00–17:00

Time	Session
14:00–14:20	Lead Instructor reviews proctor's notes on flagged questions and laptop issues
14:20–15:30	Walk through Parts A, B, and C as a group — solve the trickiest 2-3 tasks live on the projector
15:30–15:45	Break
15:45–16:45	Targeted refresher on the weakest topic identified from the morning (instructor's judgement)
16:45–17:00	Brief look ahead to Tuesday (Performance Tuning) and Wednesday (ETL/ELT & Governance)

Instructor note

Resist the temptation to re-teach everything from scratch. The point of this afternoon is targeted repair, not a repeat of three days of content. If the group is broadly strong, spend more time on the live walkthrough and less on the refresher — read the room.

Learning outcome

Participants demonstrate independent command of the first three technical days' material. The Lead Instructor identifies specific gaps to address before the final two technical days. This is a checkpoint for the programme, not a pass/fail gate for participants.

WEEK 2
TUE 30
JUN

Performance Tuning & Database Optimisation

Official theme: Performance Tuning & Database Optimisation

Programme objectives addressed: #2 SQL Development & Query Optimisation | #4 Performance Tuning

Important

SCHEDULE CHANGE — this content has moved from Monday to Tuesday so that Monday 29 June can run as the Mid-Training Assessment Day. No content has changed — only the date.

Instructor note

This day has the highest 'wow factor' — the EXPLAIN ANALYZE demonstration always gets a reaction.

Run the slow query first. Let it run. Then add the index. Then run again.

The before/after speed difference is memorable for non-technical participants.

Keep the capacity planning section grounded in NSR numbers: 12M households, 50M+ payment rows.

If yesterday's assessment surfaced a gap in indexing or EXPLAIN concepts, this day naturally reinforces them — flag the connection explicitly for participants.

Session 1 — Query optimisation & EXPLAIN ANALYZE 10:00–11:30

How the PostgreSQL query planner works (30 mins)

THEORY — Cost-based optimisation: why the planner guesses, and why it can guess wrong

Because SQL is declarative (established on Wednesday), PostgreSQL must independently decide HOW to execute every query — this is the job of the QUERY PLANNER. For any given query, there are usually MANY valid execution strategies that produce identical results (different join orders, different algorithms, different access methods), and the planner's job is to estimate the CPU and I/O cost of each candidate plan and choose the cheapest one it can find, without exhaustively trying every plan (which would be computationally infeasible for queries with many tables).

These cost estimates are NOT measured by actually running anything — they are PREDICTED, using statistics about your data stored in pg_statistic (the readable view is pg_stats), combined with tunable cost constants like seq_page_cost and random_page_cost. The statistics include things like: how many distinct values does this column have (n_distinct), what does the distribution of values look like (a histogram), and how correlated is physical row order with this column's value (correlation, which affects how efficient an index scan will be).

This is precisely why the planner can be **WRONG**, and why **ANALYZE** matters so much: statistics are a **SNAPSHOT** taken the last time **ANALYZE** ran (automatically via autovacuum, or manually). If you bulk-load a million rows and immediately query before autovacuum has run **ANALYZE**, the planner may still believe the table is nearly empty, and choose a plan that was optimal for the old, smaller table but is disastrous for the new, larger one.

EXPLAIN shows you the planner's chosen plan and its **ESTIMATES** — without running anything. **EXPLAIN ANALYZE** actually executes the query and shows you the real, measured behaviour alongside the original estimates. The single most valuable diagnostic skill in performance tuning is comparing 'rows=X' (estimated) against 'actual rows=Y' on each plan node: a large gap between the two is the planner's own confession that its statistics, and therefore its entire cost model for that part of the query, were wrong — which is the first thing to investigate, often before even looking at which physical operation (scan type, join type) was chosen.

Simplify for beginners	Extend for advanced learners
Think of the planner as a satnav choosing a driving route. It doesn't physically drive every possible route to check which is fastest — it estimates, using a map and known typical traffic patterns, which route SHOULD be quickest. If its map data is out of date (a new road opened, a road closed), it can confidently recommend a route that turns out to be slow. ANALYZE is updating the satnav's map data.	Discuss the planner's use of dynamic programming for join order selection on queries with many tables, and the <code>join_collapse_limit</code> / <code>from_collapse_limit</code> parameters that bound this search to keep planning time itself reasonable on complex queries. Ask advanced participants to consider: for a query joining 8 tables, how many possible join orders exist combinatorially, and why exhaustive search becomes impractical — motivating the genetic query optimiser (GEQO) that PostgreSQL falls back to beyond a configurable table count.

DEMO 5.1 — Reading an execution plan

```
\timing on

-- Query 1: NIN lookup (slow -- no index on nin)
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT h.household_code, h.head_name
FROM   nsr.households h
JOIN   nsr.individuals i ON i.household_id = h.household_id
WHERE  i.nin = 'AB1234567CD';

-- Read the plan:
-- Seq Scan on individuals: reading ALL 2M rows to find one
-- actual rows=1 but rows=2000000 scanned -- huge waste
-- actual time: probably 500ms - 2s

-- FIX: create the index
CREATE INDEX idx_individuals_nin ON nsr.individuals(nin);

-- Run again
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT h.household_code, h.head_name
```

```

FROM   nsr.households h
JOIN    nsr.individuals i ON i.household_id = h.household_id
WHERE   i.nin = 'AB1234567CD';
-- Now shows: Index Scan using idx_individuals_nin
-- actual time: < 1ms

```

Plan node	What it means	Good or bad?	How to fix
Seq Scan	Reading every row in the table	Bad for large tables with selective WHERE	CREATE INDEX on the filtered column
Index Scan	Using index to find matching rows	Good	Already optimal
Index Only Scan	All needed columns in the index itself	Excellent — no heap fetch needed	Add INCLUDE columns to the index
Hash Join	Hashing one table, probing with another	Good for large joins; needs work_mem	Increase work_mem if hash batches > 1
Nested Loop	For each outer row, scan inner	Good if inner is small or indexed	Add index on inner table join column
Sort	Explicit sorting of results	Can be avoided with index direction	Create index with matching ORDER BY

Indexing strategies (55 mins)

THEORY — The B-tree: why indexes turn $O(n)$ into $O(\log n)$

Without an index, finding a row matching a condition requires a **SEQUENTIAL SCAN** — PostgreSQL must read every single row in the table and check whether it matches.

This is an $O(n)$ operation: cost grows linearly with table size. Double the rows, double the work. For a table with 2 million individuals, finding one matching NIN means, on average, examining a million rows before finding it.

PostgreSQL's default index type, the **B-TREE** (balanced tree), is a data structure specifically designed to make lookups, range queries, and sorted retrieval all efficient simultaneously. A B-tree stores keys in a hierarchical, self-balancing tree where every node has multiple children, kept in sorted order, and the tree's **HEIGHT** (the number of levels you must traverse from root to leaf) grows only **LOGARITHMICALLY** with the number of entries. For 2 million rows, a B-tree typically has a height of only 3-4 levels — meaning a lookup requires reading 3-4 pages instead of up to a million rows. This is the $O(\log n)$ versus $O(n)$ difference that explains why the **EXPLAIN ANALYZE** demonstration shows milliseconds instead of seconds: it is not a small optimisation, it is a different **ALGORITHMIC COMPLEXITY CLASS** entirely.

This efficiency is not free: every index must be maintained on every **INSERT**, **UPDATE**,

and DELETE — the B-tree structure must be kept balanced and sorted as data changes, which is real, ongoing write overhead. This is the fundamental space-time trade-off of indexing: you spend extra disk space and extra write-time cost, in exchange for dramatically faster reads. The art of indexing is creating exactly the indexes your actual query workload needs, and no more — every unused index is pure overhead with zero benefit.

Beyond the basic full-table B-tree, PostgreSQL supports variations that each exploit a different structural property of your data and queries. A PARTIAL index only indexes rows matching a WHERE condition, shrinking the tree (and therefore its height and disk footprint) when most queries only ever care about a subset of rows (e.g. `is_active = TRUE`). A COVERING index (using INCLUDE) stores extra, non-key columns directly in the leaf nodes, so a query can be answered entirely from the index without ever touching the underlying table — an INDEX ONLY SCAN, the fastest possible read path. A COMPOSITE index on multiple columns is only efficient for queries filtering on a PREFIX of those columns in the same order they appear in the index definition — an index on `(lga_id, pmt_score)` helps a query filtering on `lga_id` alone, or on both columns, but does NOT help a query filtering on `pmt_score` alone, because the tree is sorted primarily by `lga_id`.

Simplify for beginners	Extend for advanced learners
A B-tree index is like a well-organised phone book. Without it, finding 'Adeyemi' means reading every name from the start. With the phone book's structure (alphabetical sections, then sub-sections), you can jump almost straight to the right page in a few flips, no matter how many millions of names the book contains.	Have advanced participants reason about WHY a composite index's column ORDER matters using the phone-book analogy extended: a phone book sorted by (surname, first_name) lets you efficiently find 'all Adeyemis' OR 'Adeyemi, Chioma' specifically, but is useless for finding 'everyone named Chioma' regardless of surname — you would have to scan the whole book. Then connect this directly to designing the <code>idx_payments_household_month</code> index used in today's lab: which queries does its (household_id, payment_month DESC) order actually serve efficiently, and which similar-looking queries would it NOT help?

DEMO 5.2 — Choosing the right index type

```
-- B-TREE (default): equality, range, ORDER BY, UNIQUE
CREATE INDEX idx_households_pmt
ON nsr.households(pmt_score ASC)
WHERE is_active = TRUE;    -- PARTIAL: only active households indexed

-- Test partial index
EXPLAIN SELECT * FROM nsr.households
WHERE pmt_score < 30 AND is_active = TRUE;
-- Should show Index Scan on idx_households_pmt

-- COMPOSITE: multi-column queries
CREATE INDEX idx_payments_household_month
ON nsr.payments(household_id, payment_month DESC);
```

```

-- This index serves: look up all payments for a household, most recent first
EXPLAIN SELECT * FROM nsr.payments
WHERE household_id = 100
ORDER BY payment_month DESC;

-- COVERING (INCLUDE): avoid heap fetch entirely
CREATE INDEX idx_households_lga_cover
ON nsr.households(lga_id)
INCLUDE (household_code, head_name, pmt_score);

-- With INCLUDE, this query is an Index Only Scan:
EXPLAIN SELECT household_code, head_name, pmt_score
FROM nsr.households WHERE lga_id = 5;

-- Compare index sizes
SELECT indexname,
       pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE tablename = 'households'
ORDER BY pg_relation_size(indexrelid) DESC;

```

Session 2 — Partitioning techniques 11:45–13:00

Table partitioning for NSR scale (55 mins)

THEORY — Partitioning as a structural, not just performance, decision

Partitioning splits ONE logical table into MULTIPLE physical tables (partitions), each storing a defined subset of rows, while every query and application still addresses the table by its single logical name. PostgreSQL determines, from the partitioning key and bounds, exactly which physical partition(s) any given row belongs to, and routes inserts and queries accordingly — entirely transparent to the application.

The core performance benefit is PARTITION PRUNING: when a query's WHERE clause constrains the partitioning key (e.g. payment_month BETWEEN '2026-01-01' AND '2026-06-30'), the planner can determine, BEFORE doing any work, that only certain partitions could possibly contain matching rows, and skips scanning the rest entirely. For a 720-million-row payments table split into 60 monthly partitions (5 years), a query for one month's data only ever touches 1/60th of the physical data — turning what would be an enormous scan into a scan of a much smaller table.

This interacts directly with everything covered in Indexing earlier today: each partition has its OWN, separate B-tree indexes (an index defined on the parent table is automatically created on every partition). A B-tree index on a 12-million-row monthly partition is shallower, faster to scan, and faster to maintain than the same index would be on a 720-million-row unpartitioned table — the $O(\log n)$ benefit of B-trees becomes even more favourable when n itself (rows per partition) is kept structurally smaller.

Partitioning also transforms two operational problems that are difficult on huge unpartitioned tables into trivial ones. ARCHIVING old data: rather than running a slow, lock-heavy DELETE FROM payments WHERE payment_month < '2020-01-01' against a live, enormous table, you can DETACH the entire 2019 partition — an near-instant metadata operation — and then drop or archive it separately, with no impact on

ongoing queries against current data. AUTOVACUUM EFFICIENCY: vacuum operates per-partition rather than against one monolithic table, so the dead-tuple cleanup discussed on Tuesday's MVCC theory runs in smaller, faster, more frequent passes, rather than occasional very long vacuum runs against the entire historical dataset.

Simplify for beginners	Extend for advanced learners
Imagine archiving paper records by YEAR into separate filing cabinets, rather than one giant cabinet holding 10 years of paperwork. If someone asks for June 2026 records, you go straight to the 2026 cabinet and ignore the other nine entirely. When 2019's retention period ends, you remove that whole cabinet — you don't have to search through everything to pull out just the 2019 papers.	Discuss the difference between RANGE, LIST, and HASH partitioning strategies and when each is appropriate (RANGE for the time-series NSR payments use case; LIST for discrete categories like partitioning by state_code; HASH when there is no natural range/list key but you still want to spread write load across multiple physical partitions). Also discuss sub-partitioning (partitioning a partition again, e.g. by month then by state) and the point at which partition COUNT itself becomes a planning-time overhead that can outweigh pruning benefits.

DEMO 5.3 — Range partitioning on payment_month

```
-- Create partitioned version of payments table
CREATE TABLE nsr.payments_by_month (
  payment_id      BIGSERIAL,
  household_id    BIGINT      NOT NULL,
  payment_month   DATE        NOT NULL,
  amount_ngn      NUMERIC(12,2) NOT NULL CHECK (amount_ngn > 0),
  payment_channel TEXT        NOT NULL,
  status          TEXT        NOT NULL DEFAULT 'pending'
) PARTITION BY RANGE (payment_month);

-- Create partitions
CREATE TABLE nsr.payments_2025
  PARTITION OF nsr.payments_by_month
  FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');

CREATE TABLE nsr.payments_2026
  PARTITION OF nsr.payments_by_month
  FOR VALUES FROM ('2026-01-01') TO ('2027-01-01');

CREATE TABLE nsr.payments_default
  PARTITION OF nsr.payments_by_month DEFAULT;

-- Partition pruning in action
EXPLAIN
SELECT * FROM nsr.payments_by_month
WHERE payment_month BETWEEN '2026-01-01' AND '2026-06-30';
-- 'Partitions selected: 1 out of 3' -- only 2026 partition scanned

-- Index on a partitioned table (applies to all partitions)
CREATE INDEX ON nsr.payments_by_month(household_id);
```

Performance monitoring (35 mins)

DEMO 5.4 — Key monitoring queries

```
-- Cache hit ratio (target: > 99%)
SELECT
    SUM(heap_blks_hit) AS cache_hits,
    SUM(heap_blks_read) AS disk_reads,
    ROUND(100.0 * SUM(heap_blks_hit)
        / NULLIF(SUM(heap_blks_hit) + SUM(heap_blks_read),0), 2) AS cache_hit_pct
FROM pg_statio_user_tables
WHERE schemaname = 'nsr';

-- Slowest queries (requires pg_stat_statements)
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
SELECT LEFT(query,80) AS query,
    calls,
    ROUND((total_exec_time/calls)::NUMERIC,1) AS avg_ms,
    ROUND(total_exec_time::NUMERIC,0) AS total_ms
FROM pg_stat_statements
WHERE calls > 10
ORDER BY avg_ms DESC LIMIT 10;

-- Index usage (find unused indexes)
SELECT schemaname, tablename, indexname,
    idx_scan AS times_used,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE schemaname = 'nsr'
ORDER BY idx_scan ASC;

-- Table bloat (dead tuples)
SELECT relname, n_live_tup, n_dead_tup,
    ROUND(n_dead_tup*100.0/NULLIF(n_live_tup+n_dead_tup,0),1) AS pct_dead
FROM pg_stat_user_tables
WHERE schemaname = 'nsr'
ORDER BY n_dead_tup DESC;
```

Session 3 — Capacity planning & optimisation lab 14:00–17:00

Capacity planning for NSR growth (30 mins)

NSR growth scenario	Current	3 years	5 years	Key implication
Active households	12.4M	16M	20M	Partitioning essential by year 3
Individuals (@ 4/household)	50M	64M	80M	Index on NIN must stay in shared_buffers
Payments (@ 12/year)	149M	192M+	240M+	Partition pruning = 80% query speedup
Concurrent users (caseworkers)	200 est.	500	800	PgBouncer connection pooling required
Database size (estimated)	80 GB	200 GB	400 GB	Storage planning + index sizing critical

Learning outcome

Participants learn how to improve database performance and manage large-scale datasets efficiently.

Intermediate: can read a basic EXPLAIN plan, create B-tree and partial indexes, use `pg_stat_statements`.

Advanced: can design a partitioning strategy, interpret cache hit ratios, and produce a capacity plan.

WEEK 2
WED 1 JUL**ETL/ELT, Data Migration & Data Governance**

Official theme: Merged day — ETL/ELT Processes & Data Migration + Data Governance & Enterprise Data Management

Programme objectives addressed: #5 ETL/ELT | #6 Data Governance | #8 Digital Welfare Administration | #10 Data-Driven Decision Making | #11 NSR Modernisation | #12 Public Sector Digital Transformation

Important

SCHEDULE CHANGE — read before preparing this session

This day **MERGES** the original Tuesday (ETL/ELT & Migration) and Wednesday (Governance) content into one day, to make room for Monday's Mid-Training Assessment.

This is the final technical day of the programme — pace deliberately. If the day runs long, the recommended cut is the postgres_fdw live demo (Session 3) — describe it conceptually from the slide instead of running it live; everything else should stay.

Instructor note

This is the most operationally relevant day for participants who manage data pipelines, and the synthesis day where technical and policy participants find common ground.

The NSR migration scenario is deliberately realistic — use the real state-federal challenge.

The NSR future-state architecture workshop in the afternoon is the most substantive group discussion of the programme — protect this time.

Session 1 — ETL/ELT frameworks & the NSR migration challenge

10:00–11:30

ETL vs ELT: choosing the right approach (25 mins)

THEORY — Why ELT became viable: pushing transformation into the database engine

ETL (Extract, Transform, Load) and ELT (Extract, Load, Transform) describe the same three logical operations in a different ORDER, and that ordering difference has real architectural consequences. ETL was the dominant pattern for decades because source databases and target data warehouses were typically **UNDERPOWERED** relative to the data volumes involved — transformation had to happen in a separate, dedicated processing tier (an ETL tool or server) because neither the source nor the target database could afford the CPU and I/O cost of complex transformation logic.

Modern database engines, including PostgreSQL, can comfortably execute complex set-based transformations directly in SQL — window functions, CTEs, JSON processing — at volumes that would have required a separate tool a decade ago. ELT exploits this: load the raw data into the target database **FIRST**, completely

unmodified, into a STAGING schema, and then use the database engine's own query power to transform it into its final form. This has two structural advantages worth understanding, not just operational convenience.

First, IDEMPOTENCY and RE-RUNABILITY: because the raw, untransformed data is preserved in staging, if your transformation logic has a bug, you fix the SQL and re-run the transformation step against the SAME staged data — no need to re-extract from the source system, which is often the slowest, most fragile, and least repeatable part of any pipeline. Second, TRANSFORMATION AS DATA, not as opaque external code: SQL transformation logic lives in the same system, the same transaction model, and the same audit/lineage tracking as the data itself — directly supporting the data lineage and governance work later today, since every transformation step can itself be logged as a database operation.

This is not a universal argument for ELT over ETL — if a transformation genuinely requires capabilities outside SQL (complex machine learning feature engineering, calling external APIs mid-transformation, certain text/NLP processing), a dedicated ETL tool or processing layer is still the right choice. The architectural question is always: where does this specific transformation run most naturally and most auditably, and for a relational, constraint-heavy domain like the NSR, that is overwhelmingly inside PostgreSQL itself.

Simplify for beginners	Extend for advanced learners
ETL is like washing and chopping vegetables at the farm before the truck delivers them to the kitchen. ELT is like delivering the vegetables exactly as picked, then washing and chopping them in the kitchen, where there are now excellent knives and running water. Either can work — it depends on whether the kitchen (the database) is well-equipped enough to do the prep itself.	Discuss the Modern Data Stack concept of a transformation layer like dbt sitting on top of ELT-staged data, applying version-controlled, tested, documented SQL transformations with explicit dependency graphs between models. Connect this to today's deduplication SQL: in a dbt-managed NSR pipeline, that ROW_NUMBER() logic would live in a version-controlled model file, automatically re-run whenever upstream staging data changes, with built-in tests asserting (for example) zero duplicate household_codes in the output.

Aspect	ETL (Extract-Transform-Load)	ELT (Extract-Load-Transform)	NSR recommendation
Where transform happens	External tool (e.g. Talend, Informatica)	Inside PostgreSQL using SQL	ELT for NSR: leverage PostgreSQL power
When to transform	Before loading	After loading into staging	ELT allows re-transformation without re-extract

Best for	Complex transformations, multiple targets	Large volumes, SQL-literate teams	NSR teams with SQL skills (after this programme)
Tool examples	Talend, Informatica, SSIS	dbt, PostgreSQL functions, stored procedures	dbt + PostgreSQL for NSR

The NSR data migration challenge (25 mins)

Nigeria's NSR migration landscape

Source 1: State Social Register files — Excel/CSV from 36 states, inconsistent formats.

Source 2: NASSP household survey data — tablet-collected, multiple survey rounds.

Source 3: NIMC NIN database — identity verification linkage.

Source 4: CBN NIBSS payment records — historical payment files.

Key challenge: the same household appears in multiple sources under different codes.

DEMO 7.1 — The staged migration pattern

```
CREATE SCHEMA nsr_staging;

CREATE TABLE nsr_staging.households_raw (
  source_file TEXT, source_state TEXT, household_code TEXT, head_name TEXT,
  lga_name TEXT, state_name TEXT, ward TEXT,
  num_members TEXT, pmt_score TEXT, enumerated_at TEXT, nin TEXT,
  loaded_at TIMESTAMPTZ DEFAULT now()
);

-- Data quality validation
SELECT
  COUNT(*) AS total_rows,
  COUNT(*) FILTER (WHERE num_members !~ '^[0-9]+$') AS invalid_members,
  COUNT(*) FILTER (WHERE pmt_score !~ '^[0-9]+\.[0-9]*$') AS invalid_scores,
  COUNT(*) FILTER (WHERE state_name NOT IN
    (SELECT state_name FROM nsr.states)) AS unknown_states
FROM nsr_staging.households_raw;
```

Migration planning framework & deduplication (25 mins)

Phase	Activity	Quality gate
1. Inventory -> 2. Stage	Profile sources; load raw data without transformation	Every source row captured; no rejection
3. Validate -> 4. Transform	Identify quality issues; apply business rules	< 5% invalid rows before proceeding
5. Reconcile -> 6. Dedup	Verify row counts; resolve duplicates	Zero unaccounted rows; zero duplicate keys

DEMO 7.2 — Deduplication and reconciliation

```
WITH deduped AS (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY household_code
    ORDER BY enumerated_at DESC NULLS LAST, loaded_at DESC
  ) AS rn
  FROM nsr_staging.households_raw
)
```

```

) AS rn
FROM nsr_staging.households_raw
WHERE num_members ~ '^[0-9]+$'
    AND state_name IN (SELECT state_name FROM nsr.states)
)
INSERT INTO nsr.households
    (household_code, head_name, lga_id, ward, num_members, pmt_score, enumerated_at)
SELECT d.household_code, TRIM(d.head_name), l.lga_id, TRIM(d.ward),
    d.num_members::SMALLINT, d.pmt_score::NUMERIC(6,3), d.enumerated_at::DATE
FROM    deduped d
JOIN    nsr.lgas l ON LOWER(TRIM(l.lga_name)) = LOWER(TRIM(d.lga_name))
WHERE   d.rn = 1
ON CONFLICT (household_code) DO UPDATE
    SET pmt_score = EXCLUDED.pmt_score
    WHERE nsr.households.enumerated_at < EXCLUDED.enumerated_at;

-- Reconciliation: staging vs loaded row counts
SELECT
    (SELECT COUNT(*) FROM nsr_staging.households_raw) AS staging_rows,
    (SELECT COUNT(*) FROM nsr.households)              AS loaded_rows;

```

Session 2 — Data governance frameworks & NDP Act 2023 11:45–13:00

What data governance means for the NSR (30 mins)

THEORY — Data governance as accountable stewardship, not bureaucracy

Data governance is frequently misunderstood as a paperwork exercise. Properly defined, it is the set of policies, roles, and CONTROLS that ensure data is accurate, available to the right people (and only the right people), and auditable — that is, that you can prove, after the fact, who did what to which data and when. The word 'controls' is doing the real work in that definition: governance that exists only as a written policy document, with no technical enforcement, is not governance — it is a wish. Everything covered this week — CHECK constraints, RBAC, Row-Level Security, pgAudit, the audit_log trigger — exists specifically to convert governance POLICY into governance ENFORCEMENT.

This distinction matters because of a structural property of large organisations: people change roles, staff turn over, institutional memory of 'why we do it this way' fades, and informal practices drift over time even when everyone has good intentions. A policy that says 'caseworkers should only access their own state's data' is a SUGGESTION that erodes under pressure (a helpful colleague checking a record 'just this once' for someone in another state). A Row-Level Security policy that says the same thing is a STRUCTURAL FACT about what the database will and will not return, regardless of anyone's intentions, good or bad, on any given day.

The six governance dimensions in the table below are not arbitrary — they map onto a well-established pattern in data management literature (data quality, security, lifecycle/retention, provenance/lineage, and interoperability), and each one maps onto a SPECIFIC, already-built PostgreSQL mechanism from this week. This is the most important idea to leave participants with today: data governance for the NSR is not a new initiative requiring new tools — it is the deliberate, systematic APPLICATION of everything already covered this week, organised around the question 'what could go wrong with this data, and what technical control prevents it structurally,

rather than relying on hoping everyone follows the rules?'

Simplify for beginners	Extend for advanced learners
A 'No Smoking' sign is a policy. A smoke detector connected to a sprinkler system is a control. Good data governance is mostly about installing detectors and sprinklers — not just putting up signs and hoping everyone reads them and complies every single time, for years, without exception.	Discuss the difference between PREVENTIVE controls (CHECK constraints, RLS — stop a problem before it happens), DETECTIVE controls (pgAudit, the audit_log trigger — notice a problem after it happens), and CORRECTIVE controls (a documented incident-response procedure for a data breach under NDP Act Article 5). A mature governance programme needs all three categories; ask advanced participants which category each of today's six PostgreSQL controls falls into, and whether the NSR currently has any corrective controls defined at all outside the database layer.

Governance dimension	PostgreSQL control	NSR example
Data quality	CHECK constraints, triggers, validation rules	NIN format enforcement; PMT score range check
Access control	RBAC + Row-Level Security	Caseworker sees only their state's households
Audit trail	pgAudit + audit_log trigger	'Who changed this household's PMT score on 15 June?'
Data retention	Partitioning + scheduled DROP; pg_cron	Old payment records archived after 7 years
Lineage	Staging schema + migration logs	Track which survey round produced each household record
Interoperability	Views, foreign data wrappers, logical replication	DWP-equivalent: share eligible household list with CBN

NDP Act 2023 and PostgreSQL (35 mins)

NDP Act 2023 Article	Requirement	Database implementation
Art 2.5 — Lawful basis	Every access has a documented lawful basis	pgAudit session logging; audit_log trigger
Art 2.6 — Minimum access	Personnel access only data necessary for their role	RBAC roles + Row-Level Security policies
Art 2.7 — Data accuracy	Personal data accurate and up to date	CHECK constraints, NOT NULL, enumeration_round tracking

Art 2.8 — Storage limitation	No longer than necessary for the purpose	Partition by year; automate archival with pg_cron
Art 3.1 — Subject access	Data subjects can request their own data	Query by household_id returning all records
Art 5.1 — Technical measures	Appropriate technical security measures	TLS + pgcrypto + RLS + pgAudit

Metadata, lineage & master data management (40 mins)

DEMO 7.3 — Metadata catalogue and lineage table

```
-- All tables with row counts and sizes
SELECT schemaname, tablename,
       pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS
total_size,
       pg_stat_user_tables.n_live_tup AS approx_rows
FROM   pg_tables
JOIN   pg_stat_user_tables USING (schemaname, tablename)
WHERE  schemaname = 'nsr'
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC;

-- Data lineage: track where each household record came from
CREATE TABLE nsr.data_lineage (
  lineage_id      BIGSERIAL      PRIMARY KEY,
  entity_type     TEXT           NOT NULL,
  entity_id       BIGINT         NOT NULL,
  source_system   TEXT           NOT NULL,
  migration_batch TEXT           NOT NULL,
  ingested_at     TIMESTAMPTZ    NOT NULL DEFAULT now(),
  ingested_by     TEXT           NOT NULL DEFAULT session_user
);
```

Master data management in a social register context

Master data: the authoritative reference records that other data links to.

MDM problem: the same household appears in 3 different state systems with 3 different codes.

MDM solution: a Golden Record — a canonical version created by matching rules, typically the same CTE + RANK()/ROW_NUMBER() pattern used in Session 1's deduplication.

Session 3 — Interoperability & NSR future-state architecture workshop 14:00–17:00

Interoperability frameworks (20 mins)

Integration partner	Data exchanged	Integration method
NIMC (NIN authority)	NIN verification, biometric confirmation	API / batch file
CBN / NIBSS	Payment authorisation, account verification	API
State Social Registers	Household enumeration data	Batch ETL (CSV/XML)
World Bank monitoring	Aggregate programme statistics	Read-only reporting schema

DEMO 7.4 — postgres_fdw for external data integration (cut first if time-pressed)

```
CREATE EXTENSION IF NOT EXISTS postgres_fdw;
CREATE SERVER kano_state_db FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (host 'kano-db.nsio.gov.ng', port '5432', dbname 'kano_register');
CREATE USER MAPPING FOR nsr_dba SERVER kano_state_db
  OPTIONS (user 'federal_reader', password 'SecurePass!');
IMPORT FOREIGN SCHEMA public LIMIT TO (households)
  FROM SERVER kano_state_db INTO nsr_staging;
SELECT COUNT(*) FROM nsr_staging.households; -- runs REMOTELY
```

NSR future-state architecture workshop (100 mins)**Instructor note**

Three groups. Each designs one component of the future-state NSR architecture.

They will present these at the programme closing ceremony.

Group 1 — Data Layer: schema, partitioning, indexing for 20M households.

Group 2 — Security & Governance: RBAC, RLS, audit, NDP Act 2023 compliance.

Group 3 — Integration: ETL pipeline, HA topology, interoperability framework.

Use the Blueprint template in the Participant Workbook.

Learning outcome

Participants understand how to migrate, integrate and maintain high-quality data across multiple systems, AND develop practical approaches to governance, compliance and enterprise data management.

Intermediate: can write a validation query, load data with COPY, map NDP Act requirements to database controls.

Advanced: can design a full migration pipeline with deduplication and reconciliation, configure postgres_fdw, and produce a governance policy.

WEEK 2
FRI 3 JUL**Programme Review & Closing Ceremony****Official theme:** Programme Review & Closing Ceremony**Status:** Afternoon programme — morning (Study Visit 3) scheduled separately, not covered in this section**Important****SCOPE NOTE**

This section covers the AFTERNOON closing ceremony only. The morning Study Visit 3 (Citizens Advice and/or NHS Digital, per the original programme) is not detailed here — confirm separately whether it proceeds, is shortened, or is replaced, and adjust the ceremony start time below accordingly. Everything in this section assumes the ceremony begins at 14:00; shift all timings uniformly if the actual start time differs.

Instructor note

This is the emotional and symbolic close of a two-week programme — it deserves real preparation, not an improvised wrap-up. Participants should leave feeling their two weeks produced something concrete (the architecture blueprints) and that their achievement is marked properly (certificates, recognition, a clear sense of closure).
Brief AUKTIE leadership and any VIP guests on the running order at least 24 hours ahead.
Print certificates the day before — do not leave this to the morning of.

Pre-ceremony checklist — complete by Thursday 2 July evening

Item	Owner	Done?
Certificates printed, names checked for spelling, signed by Lead Instructor	Lead Instructor / AUKTIE admin	
Presentation order confirmed and communicated to the three groups	Lead Instructor	
Projector/laptop tested with each group's blueprint material	AUKTIE admin	
Feedback forms printed (one per participant) or digital link ready	AUKTIE admin	
Reflection sheets printed (one per participant)	AUKTIE admin	
Refreshments/reception catering confirmed for closing slot	AUKTIE admin	

VIP guests / AUKTIE leadership briefed on running order and their speaking slot	Lead Instructor	
Group photo arrangement confirmed (photographer or staff member assigned)	AUKTIE admin	

Running order — afternoon 14:00–17:00

Time	Session
14:00–14:10	Welcome back & framing for the afternoon (Lead Instructor)
14:10–14:40	Group 1 presentation — Data Layer Architecture (10 min present + 5 min Q&A, x2 buffer)
14:40–15:10	Group 2 presentation — Security & Governance Architecture
15:10–15:40	Group 3 presentation — Integration & Operations Architecture
15:40–15:55	Break / refreshments
15:55–16:15	Study visit reflections & lessons learned (open floor, facilitated)
16:15–16:30	Knowledge transfer discussion: 'what happens when we get home?'
16:30–16:45	Certificate presentation
16:45–16:55	Closing remarks (AUKTIE leadership + Lead Instructor)
16:55–17:00	Group photo
17:00 onward	Networking reception

Section 1 — Welcome back & framing 14:00–14:10

Script — say this or adapt it

Opening remarks

'Good afternoon everyone. Two weeks ago, on Tuesday 23 June, we opened this room with a single question: what is the biggest data management problem in your organisation today? I still have your answers on the flipchart from that first morning.

This afternoon is about closing that loop. You are going to show us, in your own words and your own designs, how you would actually solve those problems using everything we've covered — PostgreSQL architecture, SQL, security, performance, governance, and the lessons from Innovation Hub, Birmingham.

Three groups will present a future-state architecture for the National Social Register. These are not abstract exercises — treat them as the first draft of a real proposal you could take back to your ministries.

Let's begin with Group 1.'

Tip

If you kept the Day 1 (Tuesday 23 June) flipchart, physically bring it into the room now and reference specific answers written on it. This callback is one of the most memorable closing techniques available to you — use it.

Section 2 — Group presentations 14:10–15:40

Each group presents the blueprint they designed during Wednesday 1 July's future-state architecture workshop (see that day's content for the original brief).

Group	Topic	What they should cover	Time
Group 1	Data Layer Architecture	Schema design for 20M households; partitioning strategy with justification; index design with justification	10 min + 5 min Q&A
Group 2	Security & Governance Architecture	Full role model with RLS policies; audit architecture; NDP Act 2023 compliance matrix	10 min + 5 min Q&A
Group 3	Integration & Operations Architecture	ETL pipeline design; HA topology with RTO/RPO targets; interoperability framework (NIN/CBN/state registers)	10 min + 5 min Q&A

Facilitation notes

- Keep a visible timer running for each group — 10 minutes goes quickly when groups are proud of their work
- Ask at least one probing question per group if the floor doesn't generate one — e.g. 'What happens to this design if a state submits data in a format you didn't anticipate?'
- Note one genuine strength of each group's design publicly before moving to Q&A — this matters more here than in a normal classroom critique
- If a group's design has a genuine technical gap (e.g. missed RLS on a sensitive table), raise it constructively as a question, not a correction: 'Talk me through who can see this table and why'

Instructor note

Optional: ask each group to email or hand over their design notes after presenting. These make an excellent appendix to a short programme completion report for AUKTIE and the Nigerian sponsoring ministry, and demonstrate concrete deliverables from the two weeks beyond attendance alone.

Section 3 — Study visit reflections & lessons learned 15:55–16:15

Facilitation approach

This is an open floor discussion, not a formal presentation. The goal is to surface what actually changed in participants' thinking across the three study visits (Innovation Hub, Birmingham, and whichever Week 2 visits took place).

Discussion prompts — use 2-3, not all of them

'What is one thing you saw on a study visit that you wish existed in your own organisation already?'

'What surprised you most about how the UK handles a problem you also face in Nigeria?'

'Which study visit changed your mind about something, even slightly?'

'If you could bring one UK official to see the NSR in action, who would it be and why?'

Section 4 — Knowledge transfer discussion 16:15–16:30

This is the most consequential 15 minutes of the ceremony — it is where the programme's impact either gets anchored into real action, or evaporates as soon as people leave the room.

Discussion prompts

'Who specifically, back in your organisation, needs to hear what you learned here?'

'What is the SINGLE most important thing you will personally do differently in the next 30 days?'

'Is there a colleague who should attend a future iteration of this programme?'

Tip

If your organisation wants a more structured outcome here, distribute the Reflection Sheet (included in the Participant Workbook) now rather than waiting — give participants 5 quiet minutes to write before opening the floor, which produces noticeably more specific and useful answers than asking people to speak from memory alone.

Section 5 — Certificate presentation 16:30–16:45**Suggested format**

28. Call participants up individually, in alphabetical order or by table, to keep pace moving
29. Say each person's full name and a one-line personal note if you can manage it (e.g. their strongest contribution, a memorable question they asked) — this takes preparation but is the single highest-value 15 minutes of the day for participant goodwill
30. Hand the certificate, allow a handshake/photo moment, brief applause, next person
31. If AUKTIE leadership or a VIP guest is present, they can hand certificates while the Lead Instructor reads names — splits the workload and adds prestige

Important

Have a SHORT card or note with each participant's name and role/organisation ready in your hand before this section starts. Do not attempt this from memory — even with good intentions, mispronouncing a name in this moment undermines the recognition this section exists to provide.

Section 6 — Closing remarks 16:45–16:55**Suggested running order**

32. AUKTIE leadership remarks (3-4 min) — thank sponsors, acknowledge organising effort, note next steps for the relationship
33. Lead Instructor closing remarks (3-4 min) — see script below

Lead Instructor closing script — say this or adapt it

'Two weeks ago none of this room could write a window function, design a Row-Level Security policy, or read an EXPLAIN ANALYZE plan. Today you can — and more importantly, you've connected every one of those skills back to a real national system that millions of people depend on.

The technology will keep changing. PostgreSQL will have a version 19, then a 20.

What doesn't change is the discipline underneath it — normalise your data, protect it with the right access controls, measure before you optimise, and never assume good intentions are a substitute for a database that enforces the rule structurally.

I've watched you build genuinely good architecture this week. Go and build it for real.

Thank you, and congratulations.'

Section 7 — Group photo & networking reception 16:55 onward

- Arrange the group photo BEFORE people start dispersing for the reception — attendance for photos drops fast once refreshments are visible
- Briefly introduce any AUKTIE staff who will remain a point of contact after the programme ends
- If a feedback form has not yet been completed, the reception is the last reliable moment to collect it — have forms or a QR code visibly available near the refreshments

Learning outcome

Participants leave with a tangible deliverable (their group's architecture blueprint), formal recognition of completion (certificate), and a personally-written commitment to specific next actions (reflection sheet) — converting two weeks of training into the start of an implementation intention rather than a pleasant memory alone.